CSC 648-848 Fall 2025 Milestone 5
Team 14
GatorTutors

Name	Role
William Wong	Team Lead
James Barr	Frontend Lead
Ethan Niu	Frontend Engineer
Uchenna Duru	Backend Lead
Randy Chen	Backend Engineer
Cyrus Molina-Perez	Github Maintainer

URL: http://18.219.122.227/
Date: 12/17/2025

Table of Contents

Table of Contents
1. Product Summary
2. Milestone Documents – M1 to M4
    Revised M1 Report:
    1. Executive Summary
    2. Personae
    3. High-Level Use Cases
    4. Main Data Items & Entities
    5. High-Level Functional Requirements
    6. Non-Functional Requirements
    7. Competitive Analysis
    8. High-Level System Architecture & Technologies
    9. Use of GenAI Tools (All, compiled by Leader)
    10. Team & Roles
    11. Team Lead Checklist
    Revised M2 Report:
    1. Executive Summary
    2. List of main data items and entities
    3. Functional Requirements
    4. UI Storyboards for each main use case
    5. High level Architecture, Database Organization summary only
    6. Identify actual key risks for your project at this time
    7. Use of GenAI tools
    8.Team Lead Checklist
    M3 Feedback Summary Report:
    M4
    Usability test plan
    QA Test Plan & QA Testing
    Peer Code Review
    Self Check on Best Practices for Security
    Self Check of the Adherence to Original Non-Functional Specs
    Use of GenAI
3. Product Screen Shots
4. Database Organization
5. GitHub Organization
6. Google Analytics Stats Plot
7. Project Management

8. Team Member Self-Assessment and Contributions

1. Product Summary

Product Name: GatorTutors

Major Functions:

- Tutoring site works exclusively for SFSU students
- Students can create an account using their student email
- Students can login to their account
- Students can search for tutors using optional filters
- Students can book tutoring sessions with an available or specific tutor
- Students can communicate with tutors through messages
- Students can view booked tutoring sessions in their dashboard
- Students can view their inbox in their dashboard
- Students can view their profile in their dashboard
- Students can apply to become a tutor
- Tutors can login to their account
- Tutors can view booked sessions in their dashboard
- Tutors can set their availability in their dashboard

Uniqueness:

- What makes our website unique is that it is specific to SFSU affiliates by ensuring members are using SFSU emails. The product also allows peer-to-peer messages between students and tutors, facilitating communication outside of tutoring sessions. Additionally, students can search for classes using a variety of optional filters.

2. Milestone Documents – M1 to M4

Revised M1 Report:

1. Executive Summary

- **Motivation**

  At San Francisco State University, tutoring is vital but often limited by rigid schedules, course mismatches, and disconnected systems. The demographic of the student base is unlike most academic institutions in the sense that many students are commuters, working parents, and graduate researchers. These groups will struggle to find tutoring help that fits their class topics and time constraints. *GatorTutors* was created to solve this problem by building a unified, student-driven tutoring platform tailored for the SFSU community.

- **What GatorTutors Does**
  GatorTutors connects SFSU students and verified peer tutors through an intuitive web interface that simplifies booking, communication, and session tracking. Students can search for tutors by course number, subject, or availability, and instantly book either online or in-person sessions. Tutors can manage their schedules, upload notes, and review session histories. The platform automatically handles notifications and reminders, ensuring smooth coordination for everyone. All interactions happen securely within the site—no third-party emails or payments required—making the system both safe and campus-compliant.
- **Why GatorTutors is Unique to SFSU**
  Unlike generic tutoring services such as Kumon.com or Mathnasium, GatorTutors is designed *for and by* SFSU students. It integrates directly with the university's course catalog, allowing tutoring to be matched at the class-code level rather than by vague subject areas. This means a Computer Science student can find a peer tutor who has already taken "CSC 415" or "CSC 648" at SFSU and understands the exact curriculum and professor expectations. The platform also verifies users through SFSU email credentials, maintaining a safe, closed network exclusive to our campus community. What's not better than someone who's been through that exact class giving exclusive tips and tricks specific to the professor's curriculum.
- **Value and Impact**
  By bridging flexibility, familiarity, and trust, GatorTutors extends the reach of academic support far beyond traditional tutoring centers. It empowers students to learn from one another, reduces administrative overhead, and creates new opportunities for paid peer mentorship.
- **About Our Team**
  We are team 14, a team composed of five dedicated SFSU computer science students. The team is led by William Wong, with James Barr and Ethan Niu leading frontend

development, Uchenna Duru and Randy Chen driving backend architecture, and Cyrus Molina-Perez managing version control.

2. Personae

Persona 1: Ryan

About:

- Graduate TA. Is a Master's student and working as a Teaching Assistant, offers tutoring as part of TA duties.
- Highly organized and driven academically, is enthused about teaching their subject matter.
- Knowledgeable in their field.

Pain Points:

- Finds logistics and repeating instructions annoying. Gets frustrated easily. Is less knowledgeable with technology, finds online sessions cumbersome.
- Prefers in-person tutoring.

Goals:

- Wants a way to easily organize recurring tutoring sessions, and wants to be easily able to share files/code snippets easily with students.
- Wants to be able to give students feedback on the platform directly.

Persona 2: Mary

About:

- Graduate student, working on a thesis research/capstone project. Lives off campus with their family (kids and spouse), and works a side job.
- Has a chaotic work-schedule due to juggling school, family, and work.
- Requires a tutor who specializes on certain topics, struggling with Biochemistry.
- Quite comfortable with technology.

Pain Points:

- Their chaotic schedule leads to only being available at late nights or at abnormal times. Is mostly only able to do online sessions.
- Chaotic schedule leads to complications setting up recurring sessions.

Goals:

- Wants a tutor to help catch up on lost time due to busy schedule.
- Be able to easily book recurring sessions with a tutor, wants flexible times, depending on their schedule.

- Wants a good way to keep track of sessions
- Prefers online sessions to fit into their schedule.

Persona 3: Alex

About:

- 18 year old, first year, full time computer science student with a packed school schedule.
- Lives in Sacramento and commutes daily by public transportation.
- Has no part time job currently.
- Doesn't have much experience with academic tutoring and online platforms.

Pain points:

- Only has basic computer and internet skills.
- Relies mostly on mobile phones and campus computers for digital access.
- Cant stay late on campus due to train schedules.
- Avoids overly technical or confusing interfaces.

Goals:

- Wants a mobile friendly interface to find and book tutors.
- Would want to see tutors that teach specific courses at SFSU. Filtering tutors based on course numbers.
- Possibility of remote sessions so they don't have to commute every time.

Persona 4: David

About:

- 22 years old, Senior in Computer Science at SFSU. Works as a tutor for specific Computer Science courses.
- Is passionate about programming and spends free time on personal coding projects.
- Known among peers for being tech-savvy and knowledgeable.
- Takes tutoring seriously and wants to give tutees a good experience, but expects them to be responsible and prepared.

Pain Points:

- Balances tutoring hours with senior-level courses and a part-time internship
- Has limited time for scheduling back-and-forth or tracking students manually
- Gets frustrated when students no-show or cancel last-minute without notice.

Goals:

- Wants a centralized platform where they can manage and accept bookings, set availability, and get reminders of appointments.
- Wants a built-in way to take notes on students and view them in future sessions.

3. High-Level Use Cases

- Use Case 1 – Filtering tutors by availability online or in-person

  Mary has an opening in her busy schedule at an abnormal time and is not able to make it to an in-person appointment. She goes to our site, and filters tutors by her time slot and whether they can do an online session. She reviews the tutors that are available according to her filters.

- Use Case 2 – Looking for course specific tutors

  Alex is struggling in his data structures class and wants to look for help via tutoring. Alex goes on our website with his phone and filters the tutors by ones that can help him with his data structures class. He looks for a timeslot that isn't too late so that he still has time to commute home afterwards.

- Use Case 3 – Managing and tracking tutoring sessions and notes

  David has finished a day of tutoring different students, and wants to keep track of future sessions he has and also write notes individually for each student. He goes to our website to see a list of all future appointments he has with each student, and he is also able to look at previous sessions that he has and is able to see the notes he took for each session.

- Use Case 4 – Booking and Receiving Session Reminders

  Mary has a busy schedule and often has trouble remembering the times of her tutoring appointments. As her appointments approach she receives reminder emails of her appointments a day before and an hour before her appointment. In the emails she is able to cancel or reschedule her appointments if she is no longer able to attend.

- Use Case 5 – Organizing Recurring In-Person Tutoring Sessions and Sharing Materials

  Ryan wants an easy way to set his tutoring hours and be able to manage sharing materials between students. He logs into our website to set his recurring availability weekly for in-person tutoring hours. After each sessions he is able to upload notes or code snippets which can be linked to the session(s) its associated to.

4. Main Data Items & Entities

- Key entities
    - Users
        - User_id (primary key)
        - First_name
        - Last_name
        - Email
        - Password
        - Role (student, user, admin)
        - Date_joined
        - Last_login
        - Bio
    - Students
        - Student_id (foreign key to user)
        - Grade_level
        - Preferred_subjects
        - Learning_goals
    - Tutors
        - Tutor_id (foreign key to user)
        - Subjects_offered
        - Schedule
        - Education_level
        - Certifications
        - Experience_years
        - Hourly_rate
        - Rating
    - Subject
        - Subject_id (primary key)
        - Subject_name
        - Category
        - Description
    - Courses
        - Course_id (primary key)
        - Tutor_id (foreign key)
        - Subject_id (foreign key)
        - Course_title
        - Description
        - Difficulty_level (beginner, intermediate, advanced)
        - Price
        - Start_date
        - End_date
        - Max_students
        - Status (upcoming, ongoing, completed)

- Enrollment
  - Enrollment_id (primary key)
  - Course_id (foreign key)
  - Student_id (foreign key)
  - Progress_percentage
  - Completion_status
- Session
  - session_id (primary key)
  - course_id (foreign key, nullable for one-on-one sessions)
  - tutor_id (foreign key)
  - student_id (foreign key)
  - subject_id (foreign key)
  - start_time, end_time
  - status (scheduled, completed, canceled)
  - session_link
  - Session_notes
- Booking/Requests
  - booking_id (primary key)
  - student_id (foreign key)
  - tutor_id (foreign key)
  - subject_id (foreign key)
  - requested_time
  - status (pending, confirmed, declined)
  - message_to_tutor
- Transactions / Billing
  - payment_id (primary key)
  - payer_id (foreign key to User)
  - receiver_id (foreign key to Tutor/User)
  - session_id (foreign key, nullable)
  - course_id (foreign key, nullable)
  - amount
  - payment_method
  - payment_status
  - transaction_date
- Review
  - review_id (primary key)
  - student_id (foreign key)
  - tutor_id (foreign key)
  - course_id (foreign key, optional)
  - session_id (foreign key, optional)
  - rating
  - comment
  - Date_posted

    - Message/Chat
        - message_id (primary key)
        - sender_id (foreign key)
        - receiver_id (foreign key)
        - timestamp
        - message_text
        - is_read
    - Availability
        - availability_id (primary key)
        - tutor_id (foreign key)
        - day_of_week
        - start_time, end_time
        - Is_booked
    - Notifications
        - notification_id (primary key)
        - user_id (foreign key)
        - type (booking, payment, message, course)
        - content
        - timestamp
        - is_read
- Permissions (unregistered, registered, admin)
    - Unregistered
    - Registered
    - Tutors
    - Admin

- Any SFSU-specific entities
    - Professors

5. High-Level Functional Requirements

5.1 Unregistered User Requirements

1. The system shall allow unregistered users to browse available tutoring subjects and courses offered at SFSU.
2. The system shall allow unregistered users to view tutor profiles including their qualifications, ratings, and availability.
3. The system shall allow unregistered users to search for tutors by subject, course number, availability, and rating.
4. The system shall require users to create an account and verify SFSU affiliation using verification code emails sent from a gmail account before booking tutoring sessions.

5.2 Registered User Requirements

5. The system shall allow registered SFSU students to book tutoring sessions with available tutors for specific SFSU courses.
6. The system shall allow students to cancel at least 24 hours in advance.
7. The system shall allow students to reschedule tutoring sessions at least 24 hours in advance.
8. The system shall allow students to view their upcoming and past tutoring sessions in a dashboard.
9. The system shall allow students to rate and review tutors after completing a session.
10. The system shall allow students to save favorite tutors for quick booking access.
11. The system shall send automated email notifications to students for session confirmations, reminders, and cancellations.
12. The system shall allow students to access session notes and materials uploaded by tutors after sessions are completed.
13. The system shall allow students to filter tutors by rating, availability, and specific SFSU course expertise.
14. The system shall allow registered SFSU tutors to create and manage their availability schedules for tutoring sessions.
15. The system shall allow tutors to specify which SFSU courses and subjects they are qualified to tutor.
16. The system shall allow tutors to upload session notes, study materials, and resources for their students.
17. The system shall allow tutors to accept or decline booking requests from students.
18. The system shall allow tutors to view their session history and student feedback.

5.3 Admin Requirements

19. The system shall allow administrators to verify and approve tutor credentials and SFSU student enrollment status.
20. The system shall allow administrators to generate reports on tutoring session metrics, tutor performance, and platform usage.
21. The system shall allow administrators to manage user accounts including suspension or removal for policy violations.

5.4 SFSU-Specific Requirement(s)

22. The system shall integrate with SFSU's course catalog to automatically populate available courses and current semester offerings for tutor specialization selection

6. Non-Functional Requirements

1. Application shall be developed, tested and deployed using tools and servers approved by Class CTO and as agreed in M0

2. Application shall be optimized for standard desktop/laptop browsers e.g. must render correctly on the two latest versions of two major browsers

3. All or selected application functions shall be rendered well on mobile devices (no native app to be developed)

4. Posting of tutor information and messaging to tutors shall be limited only to SFSU students

5. Critical data shall be stored in the database on the team's deployment server.

6. No more than 50 concurrent users shall be accessing the application at any time

7. Privacy of users shall be protected

8. The language used shall be English (no localization needed)

9. Application shall be very easy to use and intuitive

10. Application shall follow established architecture patterns

11. Application code and its repository shall be easy to inspect and maintain

12. Google analytics shall be used

13. No e-mail clients shall be allowed. Interested users (clients) can only message service providers via in-site messaging. One round of messaging (from client to service provider) is enough for this application. No chat functions shall be developed or integrated

14. Pay functionality (e.g. paying for goods and services) shall not be implemented nor simulated in UI.

15. Site security: basic best practices shall be applied (as covered in the class) for main data items

16. Media formats shall be standard as used in the market today

17. Modern SE processes and tools shall be used as specified in the class, including collaborative and continuous SW development and GenAI tools

18. The application UI (WWW and mobile) shall prominently display the following exact text on all pages "SFSU Software Engineering Project CSC 648-848, Fall 2025. For Demonstration Only” at the top of the WWW page Nav bar. (Important so as to not confuse this with a real application).

7. Competitive Analysis

Inferior: ✖ Superior: ✔

Feature	Kumon	Mathnasium	Tutor.com	SFSU Tutoring	🟣 Our Platf
Scheduling	✖ Fixed sessions	✖ Fixed weekly schedule	✔ Calendar scheduling	✖ Manual form; no weekends	✔ Flexible onlin for tutors & stud
Class specific tutoring	✖ Subject level only	✖ General skills only	✖ Subject based only	✖ Subject based only	✔ Course-code matching; peer t for matching cla
One-on-one tutoring	✖ Groups only (4–6)	✖ Group-based	✔ Private sessions available	✖ 1/week limit, few STEM tutors	✔ 1:1 sessions any subject
Online learning	✖ In-person only	✖ Minimal tech	✔ White board, chat, file share	✔ Zoom sessions available	✔ Built-in Zoom links & instant messaging
University integration	✖ Franchise	✖ Franchise	✖ No SFSU integration	✔ Connected to departments	✔ Uses SFSU c codes, verified tu hybrid on/off-can

Our SFSU Tutoring Platform directly targets the biggest weakness of existing tutoring services — a one-size-fits-all approach and an utter lack of university integration. While private competitors like Tutor.com focus on convenience, they lack campus alignment and class-level context. Kumon and Mathnasium excel at standardized learning but are totally irrelevant when it comes to higher education. SFSU's existing TASC offers strong departmental support but struggles to provide individualized scheduling, consistent STEM coverage, and any form of peer-to-peer collaboration, which in some cases can be more helpful than traditional generalized tutoring.

Our system blends the best of all worlds: real-time scheduling, class-specific matching, verified SFSU tutors, and built-in online tools for both one-on-one and group learning. By empowering students themselves to become peer tutors within their enrolled courses, our platform expands availability, reduces wait times, and enhances the sense of academic community — all while remaining tightly integrated with SFSU's systems.

8. High-Level System Architecture & Technologies

Main Software Components and Versions

- **Backend Runtime:** Node.js v20.11.1
- **Web Framework:** Express.js v4.19.2
- **Database:** MySQL 8.4.2
- **Operating System / Server Environment:** Ubuntu on AWS EC2
- **Version Control:** Git / GitHub

Deployment Cloud Service

- **Amazon Web Services (AWS)** — EC2 instance for server and database hosting

Front-End Frameworks / Technologies

- HTML / CSS / JavaScript

APIs

- Google Analytics

Supported Browsers

- **Google Chrome:** V114 - 141
- **Mozilla Firefox:** V113 - 140

9. Use of GenAI Tools (All, compiled by Leader)

The primary use of Generative AI tools occurred in Sections 1, 2, 3, 5, and 7, all during the ideation and inspiration stages of project development. AI tools were used solely to assist with brainstorming, formatting, and conceptual framing—no direct code, content, or design elements were copied without review or revision by team members.

Section 1 (Executive Summary):

AI was used to refine the project vision, clarify motivation, and structure the problem statement for clarity and conciseness.

Section 2 (Personae):

AI assisted in generating initial persona ideas, which were later refined to reflect real SFSU student demographics and tutoring use cases.

Section 3 (High-Level Use Cases):

AI supported the ideation process for realistic, scenario-based use cases. The final versions were reviewed and customized by the frontend team to ensure relevance to SFSU's academic environment.

Section 5 (Functional Requirements):

AI helped format and organize requirement statements according to standard software engineering conventions. All final requirements were written, verified, and validated by the backend team.

Section 7 (Competitive Analysis):

AI was used to brainstorm key comparison features and structure the competitive ranking table. Data and conclusions were finalized by the team after independent analysis of existing tutoring services.

Overall, AI served as a collaborative ideation partner to streamline brainstorming and improve document structure. Every section's final content was critically reviewed, rewritten, and validated by human team members to ensure accuracy, originality, and alignment with SFSU's academic standards.

10. Team & Roles

Name	Role	Email
William Wong	Team Lead	wwong23@sfsu.edu
Jamie (James) Barr	Frontend Lead	jbarr@sfsu.edu
Ethan Niu	Frontend	eniu1@sfsu.edu
Uchenna Duru	Backend Lead	uduru2@sfsu.edu
Randy Chen	Backend	rchen10@sfsu.edu
Cyrus Molina-Perez	Github Maintainer	cmolinaperez@sfsu.edu

11. Team Lead Checklist

Task	Status
So far all team members are fully engaged and attending team sessions when required	OK
Team found a time slot to meet outside of the class	DONE, every Tuesday
Team ready and able to use the chosen back and front end frameworks and those who need to learn are working on learning and practicing	DONE, using a tech stack from 317
Team reviewed class slides on requirements and use cases before drafting Milestone 1	DONE
Team reviewed non-functional requirements from “How to start...” document and developed Milestone 1 consistently	DONE
Team lead checked Milestone 1 document for quality, completeness, formatting and compliance with instructions before the submission	DONE
Team lead ensured that all team members read the final M1 and agree/understand it before submission	DONE
Team shared and discussed experience with GenAI tools among themselves	DONE
Github organized as discussed in class (e.g. master branch, development branch, folder for milestone documents etc.)	DONE

Revised M2 Report:

1. Executive Summary

- **Motivation**
  At San Francisco State University, tutoring is vital but often limited by rigid schedules, course mismatches, and disconnected systems. The demographic of the student base is unlike most academic institutions in the sense that many students are commuters, working parents, and graduate researchers. These groups will struggle to find tutoring help that fits their class topics and time constraints. *GatorTutors* was created to solve this problem by building a unified, student-driven tutoring platform tailored for the SFSU community.

- **What GatorTutors Does**
  GatorTutors connects SFSU students and verified peer tutors through an intuitive web interface that simplifies booking, communication, and session tracking. Students can search for tutors by course number, subject, or availability, and instantly book either online or in-person sessions. Tutors can manage their schedules, upload notes, and review session histories. The platform automatically handles notifications and reminders, ensuring smooth coordination for everyone. All interactions happen securely within the site—no third-party emails or payments required—making the system both safe and campus-compliant.

- **Why GatorTutors is Unique to SFSU**
  Unlike generic tutoring services such as Kumon.com or Mathnasium, GatorTutors is designed *for and by* SFSU students. It integrates directly with the university's course catalog, allowing tutoring to be matched at the class-code level rather than by vague subject areas. This means a Computer Science student can find a peer tutor who has already taken "CSC 415" or "CSC 648" at SFSU and understands the exact curriculum and professor expectations. The platform also verifies users through SFSU email credentials, maintaining a safe, closed network exclusive to our campus community. What's not better than someone who's been through that exact class giving exclusive tips and tricks specific to the professor's curriculum.

- **Value and Impact**
  By bridging flexibility, familiarity, and trust, GatorTutors extends the reach of academic support far beyond traditional tutoring centers. It empowers students to learn from one another, reduces administrative overhead, and creates new opportunities for paid peer mentorship.

- **About Our Team**
  We are team 14, a team composed of five dedicated SFSU computer science students. The team is led by William Wong, with James Barr and Ethan Niu leading frontend development, Uchenna Duru and Randy Chen driving backend architecture, and Cyrus Molina-Perez managing version control.

2. List of main data items and entities

Key entities

1. Users (Base Entity)

- user_id - Unique identifier (mandatory, auto-generated)
- first_name - User's first name (mandatory, text)
- last_name - User's last name (mandatory, text)
- email - Login email address (mandatory, unique, text)
- password - Encrypted password (mandatory, text)
- role - User type: student, tutor, admin (mandatory, selection)
- profile_picture - Path to profile image (optional, text)
- bio - Personal description (optional, text, max 500 chars)
- phone_number - Contact number (optional, text)
- date_joined - Registration date (mandatory, auto-generated)
- last_login - Last access time (optional, timestamp)
- is_active - Account status (mandatory, boolean, default true)
- sfsu_id - SFSU student/staff ID (optional, text) [SFSU-specific]
- campus_location - On/off campus (optional, selection) [SFSU-specific]

2. Students (Extends Users)

- student_id - Links to Users table (mandatory, foreign key)
- grade_level - Freshman/Sophomore/Junior/Senior/Graduate (mandatory, selection)
- major - Academic major (mandatory, text)
- preferred_subjects - Subjects seeking help in (optional, text/JSON)
- learning_goals - Personal objectives (optional, text, max 1000 chars)
- gpa - Current GPA (optional, decimal 0.0-4.0)
- sfsu_college - College within SFSU (optional, selection) [SFSU-specific]

3. Tutors (Extends Users)

- tutor_id - Links to Users table (mandatory, foreign key)
- subjects_offered - Teaching subjects (mandatory, text/JSON)
- education_level - Bachelors/Masters/PhD/Other (mandatory, selection)
- certifications - Teaching credentials (optional, text)
- experience_years - Teaching experience (mandatory, integer, default 0)
- hourly_rate - Rate per hour in USD (mandatory, decimal)
- rating - Average rating 0.00-5.00 (mandatory, decimal, auto-calculated)
- total_reviews - Number of reviews (mandatory, integer, auto-calculated)
- teaching_mode - online/in-person/both (mandatory, selection)

- sfsu_verified - Verified SFSU tutor (optional, boolean) [SFSU-specific]
- department_approved - Department approval status (optional, boolean) [SFSU-specific]

4. Subjects

- subject_id - Unique identifier (mandatory, auto-generated)
- subject_name - Name of subject (mandatory, text)
- category - STEM/Liberal Arts/Business/Other (mandatory, selection)
- description - Subject details (optional, text, max 500 chars)
- difficulty_level - beginner/intermediate/advanced (optional, selection)
- sfsu_course_code - Official SFSU code like "CS 648" (optional, text) [SFSU-specific]
- department - SFSU department (optional, text) [SFSU-specific]
- is_active - Currently offered (mandatory, boolean, default true)

5. Courses (Group Learning)

- course_id - Unique identifier (mandatory, auto-generated)
- tutor_id - Teaching tutor (mandatory, foreign key)
- subject_id - Course subject (mandatory, foreign key)
- course_title - Name of course (mandatory, text, max 200 chars)
- description - Course details (mandatory, text)
- syllabus - Course outline document (optional, text/file path)
- difficulty_level - beginner/intermediate/advanced (mandatory, selection)
- price - Total cost in USD (mandatory, decimal)
- start_date - Course start (mandatory, date)
- end_date - Course end (mandatory, date)
- schedule - Meeting times (mandatory, text/JSON)
- max_students - Capacity limit (mandatory, integer, default 10)
- current_students - Enrolled count (mandatory, integer, auto-calculated)
- status - upcoming/ongoing/completed/cancelled (mandatory, selection)
- course_materials - Links to resources (optional, text/JSON)
- location - online/on-campus/off-campus (mandatory, selection) [SFSU-specific]

6. Sessions (Individual Tutoring)

- session_id - Unique identifier (mandatory, auto-generated)
- course_id - Related course if any (optional, foreign key)
- tutor_id - Session tutor (mandatory, foreign key)
- student_id - Session student (mandatory, foreign key)
- subject_id - Session topic (mandatory, foreign key)
- start_time - Session start (mandatory, datetime)
- end_time - Session end (mandatory, datetime)
- duration_minutes - Length in minutes (mandatory, integer, auto-calculated)

- status - scheduled/completed/cancelled/no-show (mandatory, selection)
- session_type - one-on-one/group (mandatory, selection)
- session_link - Video meeting URL (optional, text)
- session_notes - Tutor's notes (optional, text)
- student_feedback - Student's notes (optional, text)
- recording_url - Session recording (optional, text)
- location - Physical location if in-person (optional, text)

7. Bookings (Session Requests)

- booking_id - Unique identifier (mandatory, auto-generated)
- student_id - Requesting student (mandatory, foreign key)
- tutor_id - Requested tutor (mandatory, foreign key)
- subject_id - Topic needed (mandatory, foreign key)
- requested_date - Preferred date (mandatory, date)
- requested_time - Preferred time (mandatory, time)
- duration_requested - Length in minutes (mandatory, integer)
- urgency - normal/urgent (mandatory, selection, default normal)
- status - pending/confirmed/declined/cancelled (mandatory, selection)
- message_to_tutor - Student's request details (optional, text, max 500 chars)
- tutor_response - Tutor's reply (optional, text)
- alternative_times - Other available slots (optional, text/JSON)

8. Enrollments

- enrollment_id - Unique identifier (mandatory, auto-generated)
- course_id - Enrolled course (mandatory, foreign key)
- student_id - Enrolled student (mandatory, foreign key)
- enrollment_date - Join date (mandatory, timestamp, auto-generated)
- progress_percentage - Course completion 0-100 (mandatory, integer, default 0)
- completion_status - enrolled/in_progress/completed/dropped (mandatory, selection)
- completion_date - Finish date (optional, timestamp)
- certificate_issued - Completion certificate (optional, boolean)

9. Reviews

- review_id - Unique identifier (mandatory, auto-generated)
- student_id - Reviewing student (mandatory, foreign key)
- tutor_id - Reviewed tutor (mandatory, foreign key)
- course_id - Related course (optional, foreign key)
- session_id - Related session (optional, foreign key)
- rating - Score 1-5 stars (mandatory, integer)
- comment - Written review (optional, text, max 1000 chars)
- date_posted - Review date (mandatory, timestamp, auto-generated)

- is_verified - Verified attendance (mandatory, boolean, auto-calculated)
- is_visible - Public visibility (mandatory, boolean, default true)
- helpful_count - Users who found helpful (optional, integer, default 0)

10. Messages

- message_id - Unique identifier (mandatory, auto-generated)
- sender_id - Sending user (mandatory, foreign key)
- receiver_id - Receiving user (mandatory, foreign key)
- conversation_id - Thread identifier (mandatory, text)
- message_text - Message content (mandatory, text)
- attachment_url - File attachment (optional, text)
- timestamp - Send time (mandatory, timestamp, auto-generated)
- is_read - Read status (mandatory, boolean, default false)
- is_deleted - Soft delete (mandatory, boolean, default false)

11. Availability

- availability_id - Unique identifier (mandatory, auto-generated)
- tutor_id - Tutor's schedule (mandatory, foreign key)
- day_of_week - Monday-Sunday (mandatory, selection)
- start_time - Available from (mandatory, time)
- end_time - Available until (mandatory, time)
- is_recurring - Weekly repeat (mandatory, boolean, default true)
- specific_date - For one-time availability (optional, date)
- is_booked - Already reserved (mandatory, boolean, default false)

12. Notifications

- notification_id - Unique identifier (mandatory, auto-generated)
- user_id - Recipient user (mandatory, foreign key)
- type - booking/payment/message/course/session/review (mandatory, selection)
- title - Notification title (mandatory, text, max 100 chars)
- content - Notification details (mandatory, text)
- related_id - ID of related item (optional, integer)
- priority - low/normal/high (mandatory, selection, default normal)
- timestamp - Created time (mandatory, timestamp, auto-generated)
- is_read - Read status (mandatory, boolean, default false)
- is_email_sent - Email sent status (optional, boolean, default false)

13. SFSU-Specific Entities

- Professor Verifications [SFSU-specific]
  - verification_id - Unique identifier (mandatory, auto-generated)
  - professor_email - SFSU professor email (mandatory, text)
  - professor_name - Professor full name (mandatory, text)

    - department - Academic department (mandatory, text)
    - verified_tutor_id - Approved tutor (mandatory, foreign key)
    - verification_date - Approval date (mandatory, timestamp)
    - comments - Professor notes (optional, text)
- Campus Resources [SFSU-specific]
    - resource_id - Unique identifier (mandatory, auto-generated)
    - resource_name - Name of resource (mandatory, text)
    - resource_type - tutoring_center/study_room/library (mandatory, selection)
    - building - Campus building (mandatory, text)
    - room_number - Location (optional, text)
    - hours_of_operation - Operating hours (mandatory, text)
    - contact_info - Phone/email (optional, text)
    - is_available - Current availability (mandatory, boolean)

3. Functional Requirements

P1

- **Unregistered Users**
    1. Users shall be allowed to create an account
    2. Users shall be able to verify SFSU affiliation
    3. The system shall allow users to view tutor profiles
    4. The system shall allow users to search tutors by categories (subject, course number, etc)
    5. Users shall be able to view available subjects
- **Registered Users**
    6. Users shall be allowed to send messages to each other
    7. Users shall be allowed to book tutoring sessions with available tutors for specific courses.

8. Users shall be allowed to view their upcoming and past tutoring sessions in a dashboard.

- **Tutors**
  9. Tutors shall be required to specify which SFSU courses and subjects they are qualified to tutor.
  10. The system shall allow tutors to accept or decline booking requests from students.
- **Admin**
  11. Admin users shall be required to approve new tutor applications
- **SFSU-Specific**
  12. The system shall integrate with SFSU's course catalog to automatically populate available courses and current semester offerings for tutor specialization selection

P2

- **Registered Users**
  13. The system shall use verification code emails sent from a gmail account before booking tutoring sessions.
  14. Users shall be allowed to cancel at least 24 hours in advance.
  15. Users shall be allowed to reschedule tutoring sessions at least 24 hours in advance.
- **Tutors**
  16. The system shall allow tutors to view their session history and student feedback.
  17. Tutors shall be allowed to create and manage their availability schedules for tutoring sessions.
- **Admin**
  18. Admin shall be allowed to verify and approve tutor credentials and SFSU student enrollment status.

P3

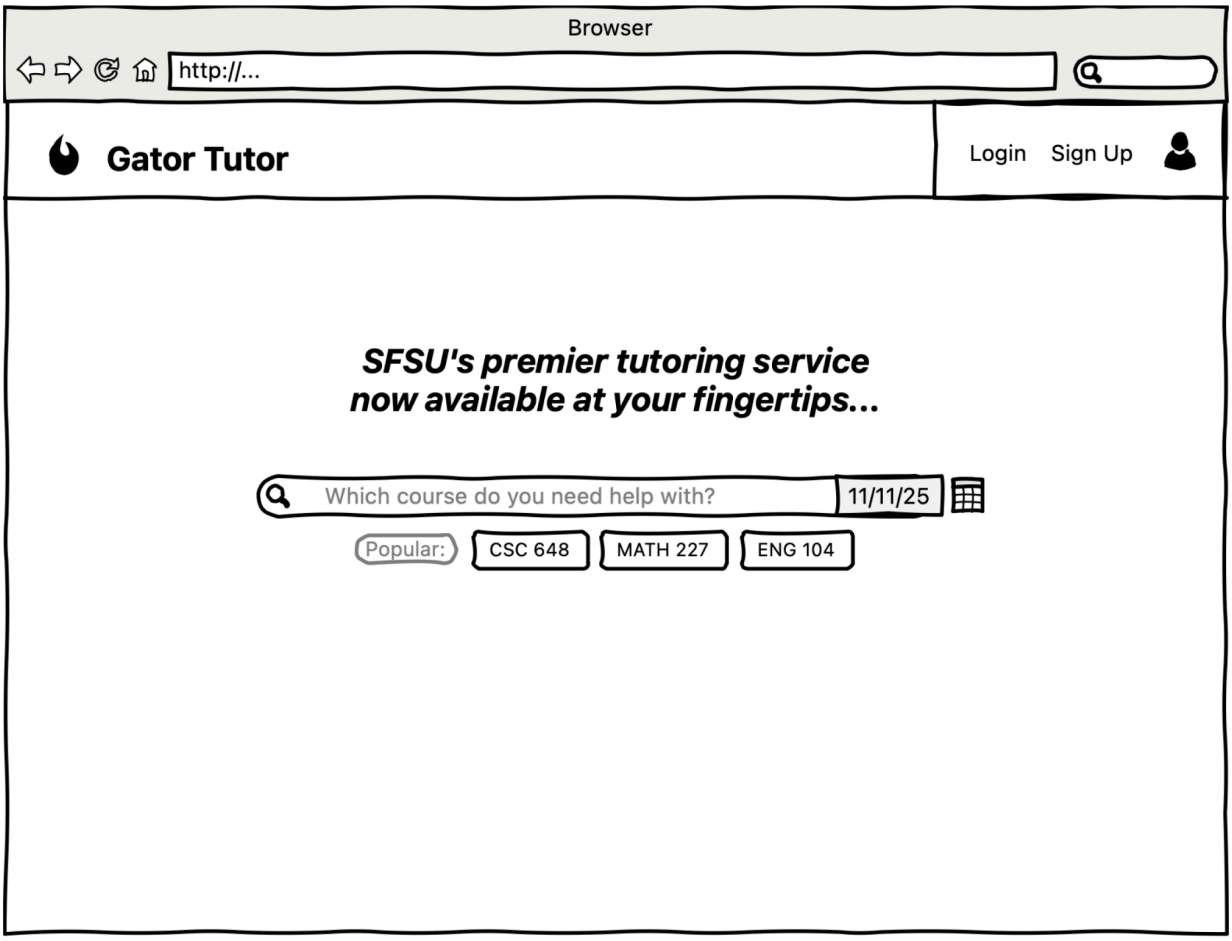
- **Registered Users**
  19. Users shall be allowed to rate and review tutors after completing a session.
  20. Users shall be allowed to save favorite tutors for quick booking access.
  21. The system shall send automated email notifications to students for session confirmations, reminders, and cancellations.
  22. Users shall be allowed to access session notes and materials uploaded by tutors after sessions are completed.
  23. Users shall be allowed to filter tutors by rating, availability, and specific SFSU course expertise.
  24. Users shall be allowed to upload session notes, study materials, and resources for their students.
- **Admin**
  25. The system shall allow administrators to generate reports on tutoring session metrics, tutor performance, and platform usage.

26. Admin users shall be required to manage user accounts including suspension or removal for policy violations.

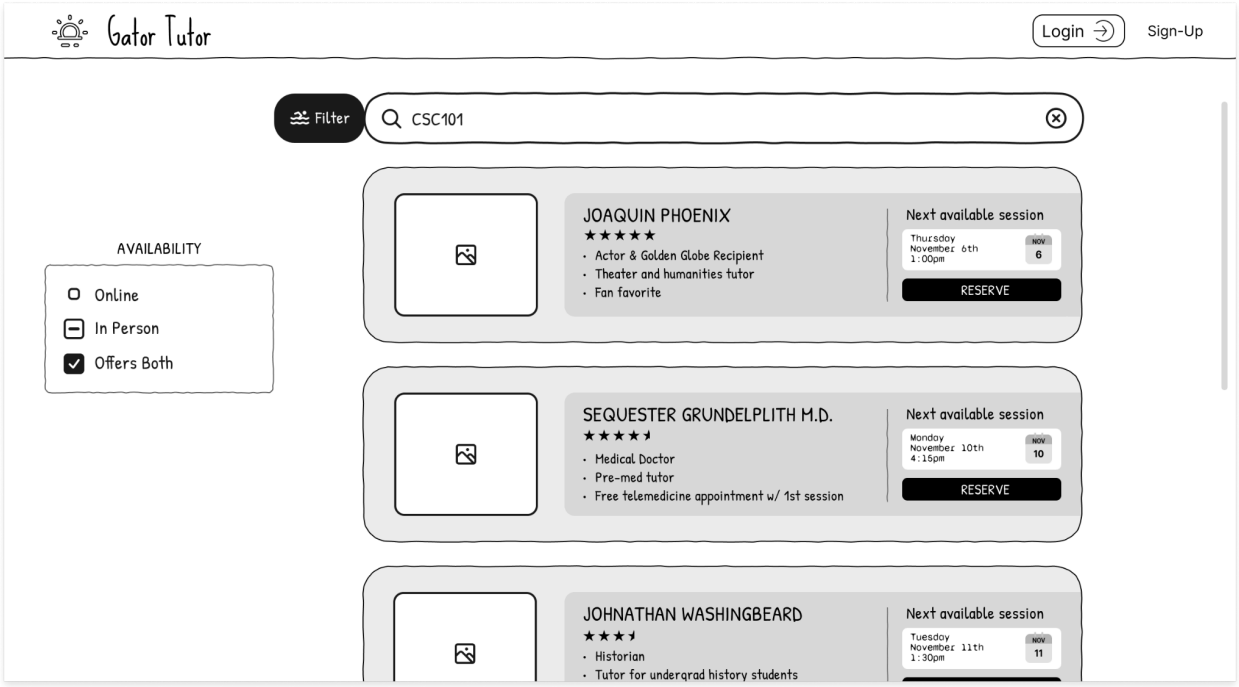
4. UI Storyboards for each main use case

User filtering tutors by availability online or in-person:

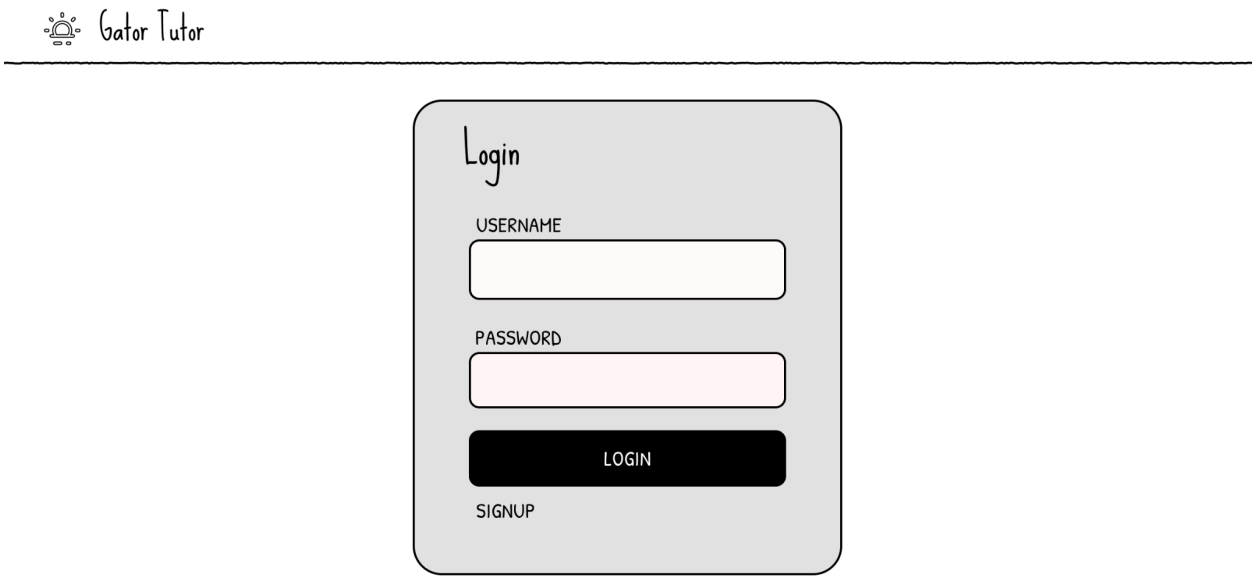
Student opens our website’s homepage



Student searches for tutors offering support in specific courses

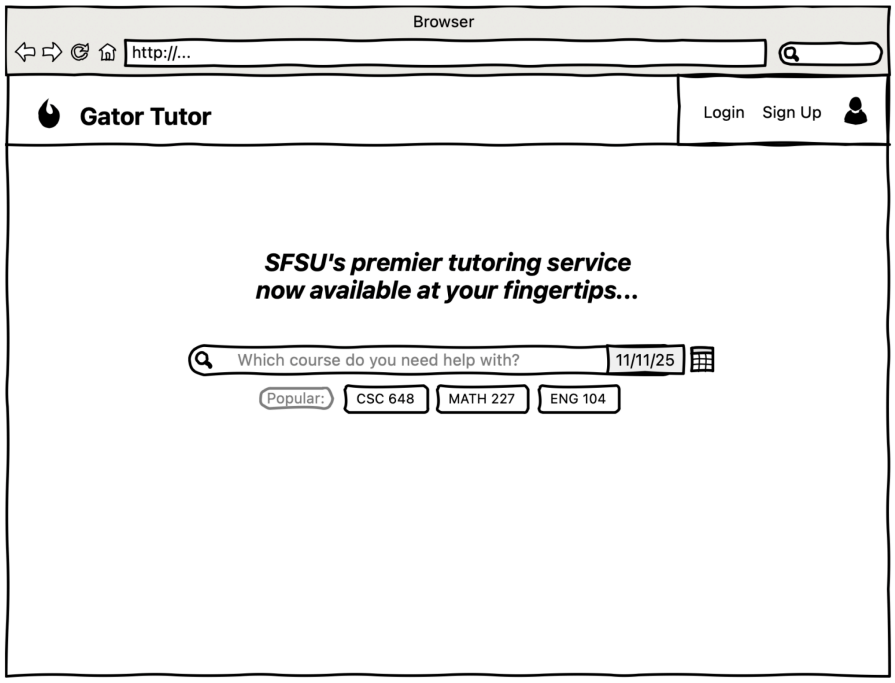


Student is prompted to login after clicking reserve

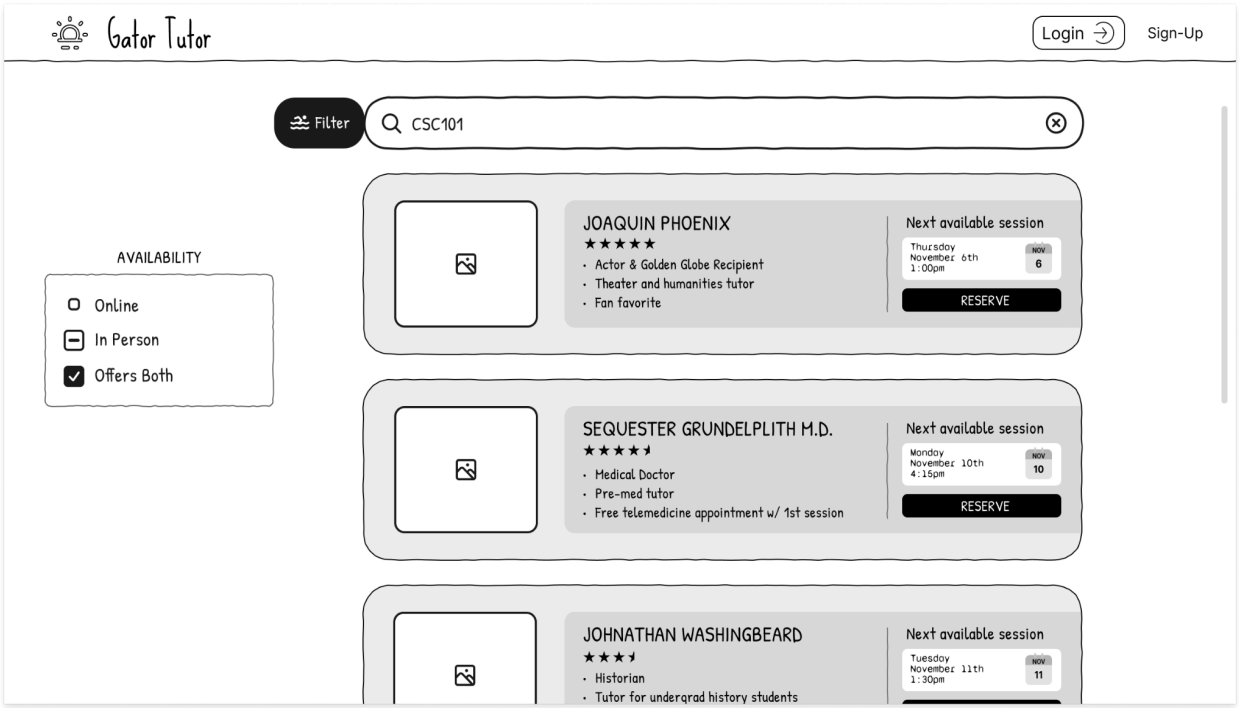


User signs up for a new account:

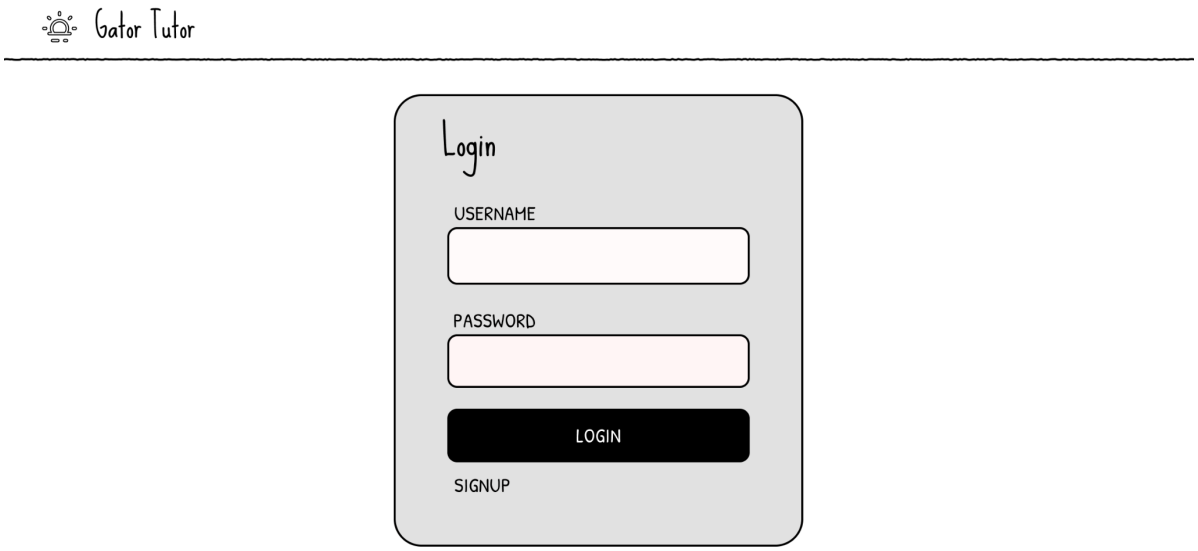
User opens our website's homepage



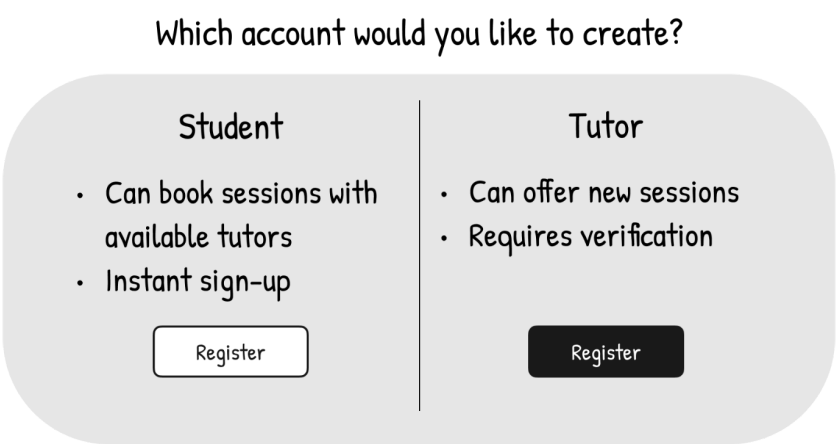
User searches for tutors offering support in specific courses



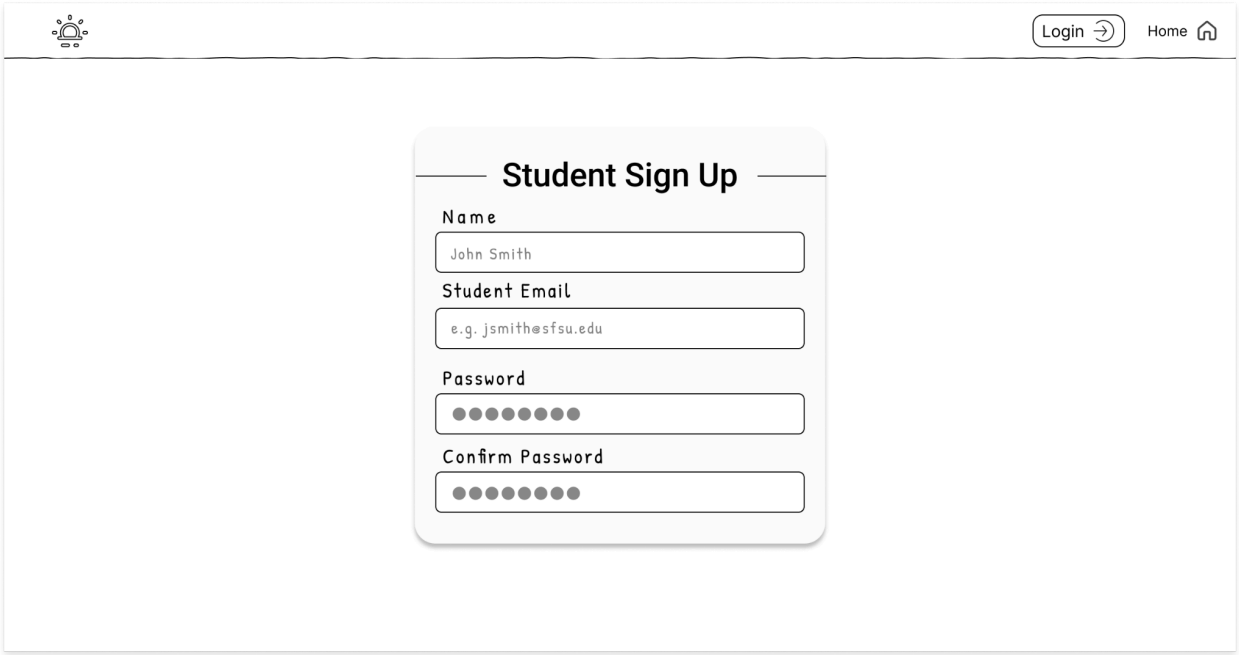
User is prompted to login after clicking reserve



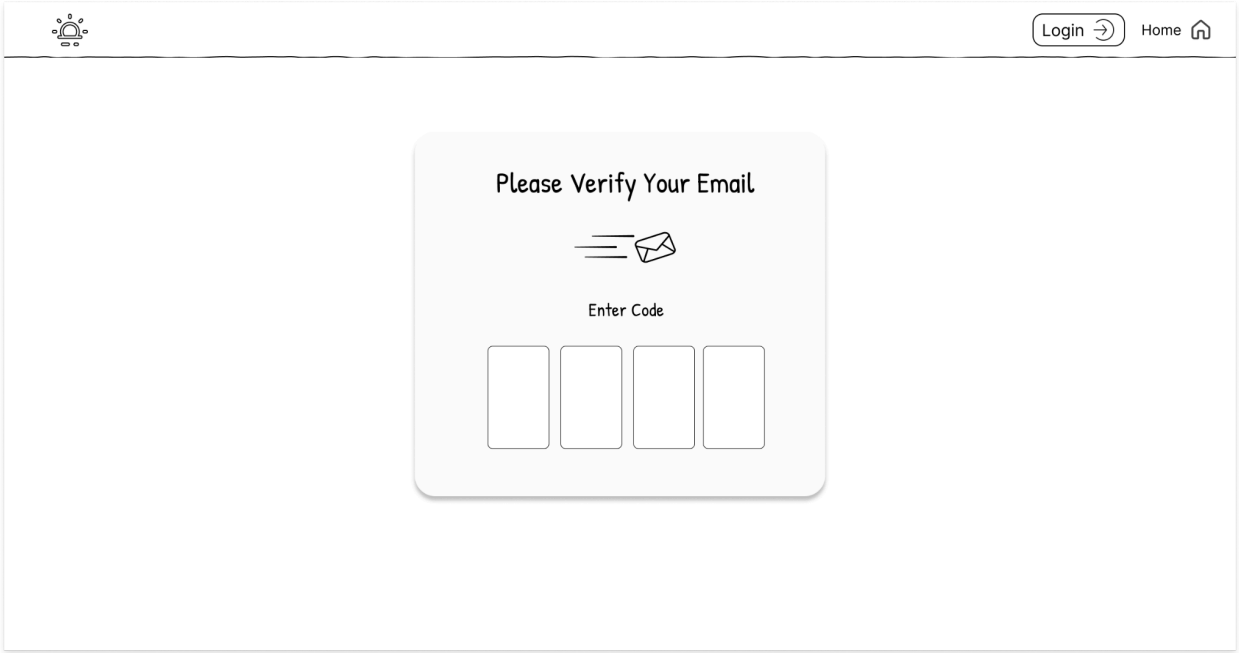
User clicks signup to make an account



User signs up a student account

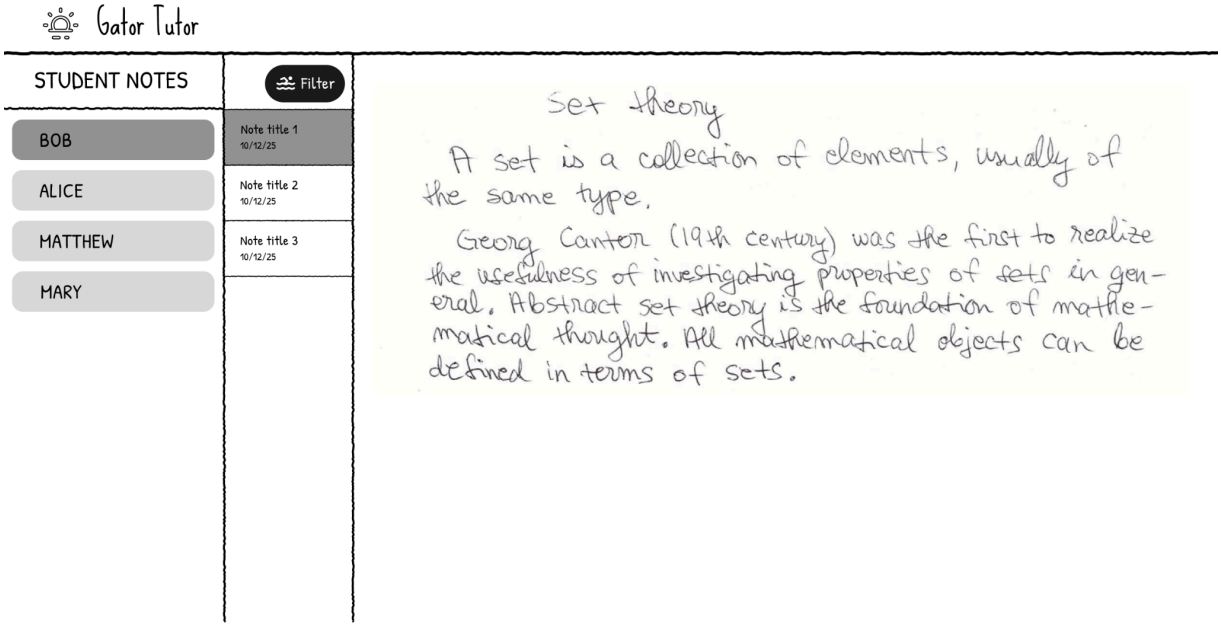


User is prompted to verify their email



Tutor looking at student notes:

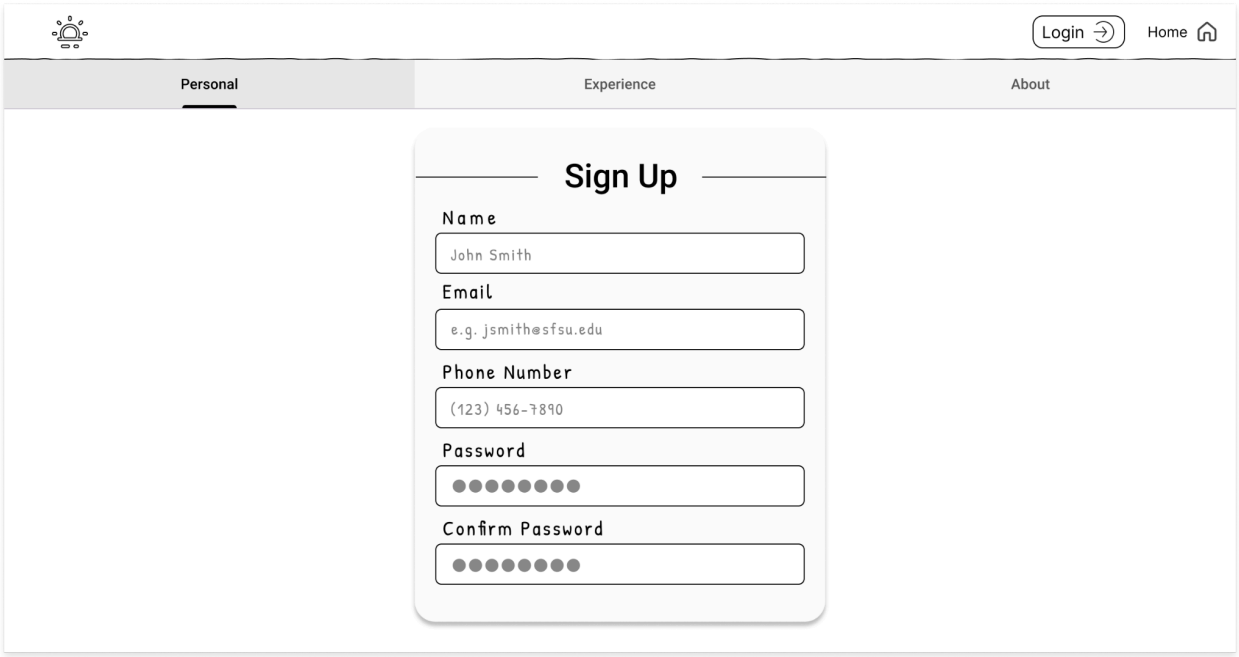
Tutor looks at student's notes for individual sessions



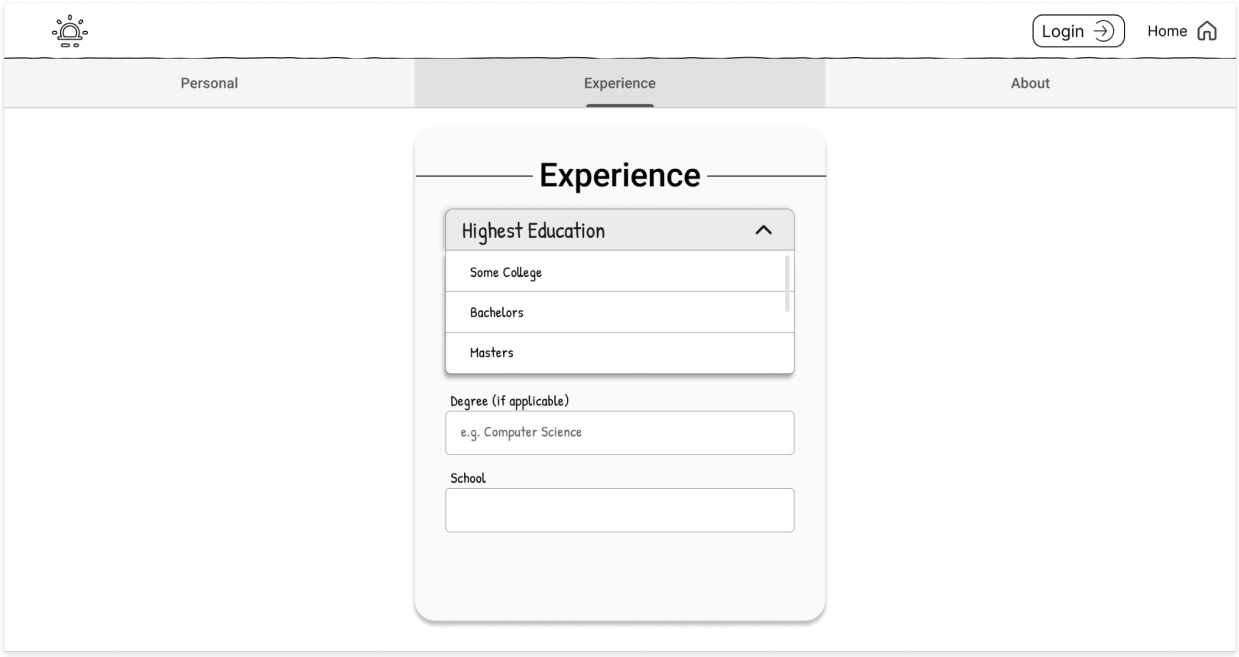
Sign Up:



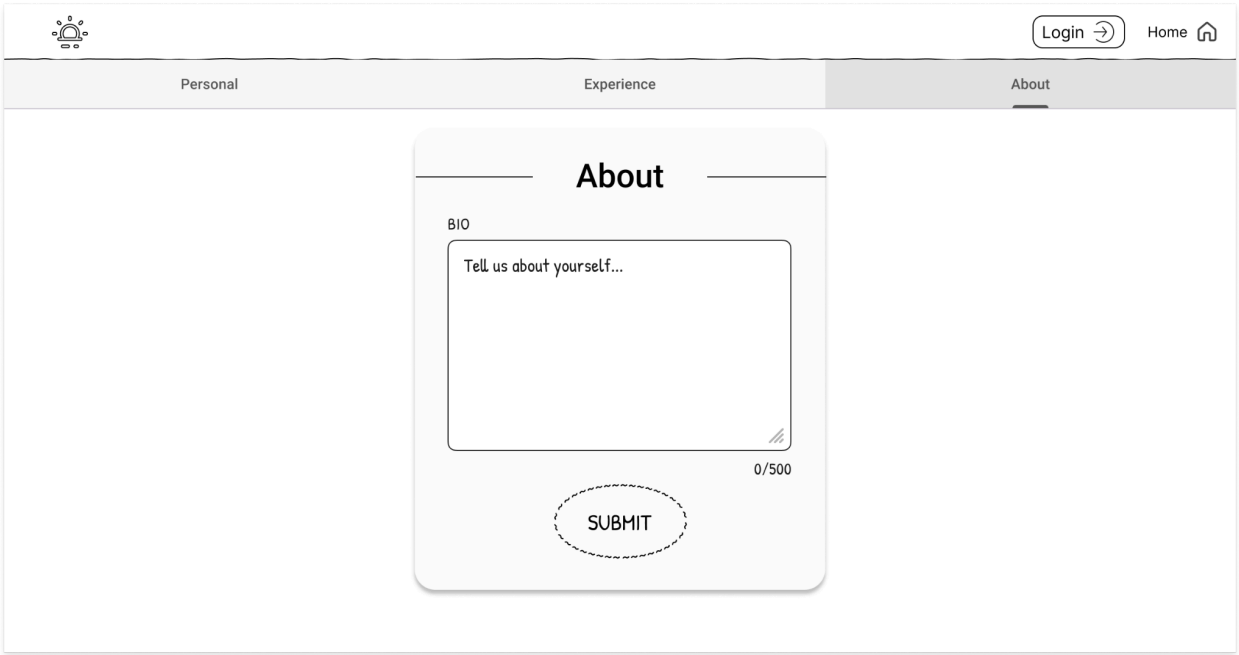
User enters account information



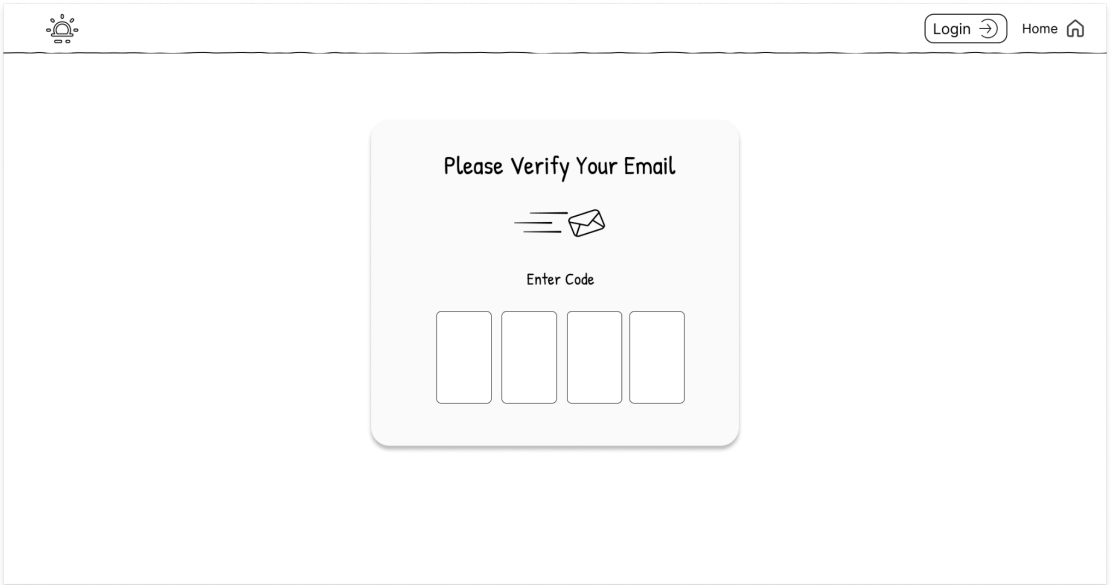
User enters their experience



User enters information about themselves



User is prompted to verify their email



5. High level Architecture, Database Organization summary only

Overall Architecture (MVC Pattern):

Model (M): Each table in MySQL is mapped to a simple model file that runs basic SQL queries (SELECT, INSERT, UPDATE, DELETE) using the mysql2 package.

View (V): HTML pages with minimal CSS and JS. Only handles basic forms and lists (e.g., login page, tutor list, course list).

Controller (C): Express routes that call model functions and return JSON or render HTML pages. Each controller handles one feature area (e.g., users, courses, sessions).

Users

- user_id (Primary key, int, auto-increment)
- first _name (varchar(50)
- last_name (varchar(50)
- email (varchar(100)
- password (varchar(255)
- role (
- profile_picture (varchar(50)
- bio (varchar(50)
- phone_number (optional, text)
- date_joined (mandatory, auto-generated)
- last_login (optional, timestamp)
- is_active (mandatory, boolean, default true)
- sfsu_id (optional, text)
- campus_location (optional, selection)

Students

- student_id (Primary Key, int, references users.user_id)
- grade_level (enum: 'freshman', 'sophomore', 'junior', 'senior', 'graduate')
- major (varchar(100), student's declared major)
- preferred_subjects (text, optional list of subjects the student is interested in)
- learning_goals (varchar(1000), optional description of academic or personal goals)
- gpa (decimal(3,2), optional student GPA) // **Remove**
- sfsu_college (varchar(100), optional college affiliation, e.g., College of Science & Engineering)

Tutors

- tutor_id (Primary Key, int, references users.user_id)
- subjects_offered (text, list of subjects the tutor can teach)

- education_level (enum: 'bachelors', 'masters', 'phd', 'other')
- certifications (text, optional tutor certifications or credentials)
- experience_years (int, number of years of tutoring or teaching experience)
- hourly_rate (decimal(10,2), tutor's hourly charge in USD)
- rating (decimal(3,2), average rating based on student reviews, default 0.00)
- total_reviews (int, total number of reviews received, default 0)
- teaching_mode (enum: 'online', 'in_person', 'both')
- sfsu_verified (tinyint(1), indicates if tutor is verified by SFSU faculty)
- department_approved (tinyint(1), marks if tutor has departmental approval)

Subjects
- subject_id (Primary Key, int, auto-increment)
- subject_name (varchar(100), name of the subject or course)
- category (enum: 'STEM', 'Liberal Arts', 'Business', 'Other') // **Specify more, stem is too broad (Go at department level not every major)**
- description (varchar(500), optional short summary of the subject)
- difficulty_level (enum: 'beginner', 'intermediate', 'advanced')
- sfsu_course_code (varchar(20), optional university course reference)
- department (varchar(100), academic department offering the subject)
- is_active (tinyint(1), default 1, marks if the subject is active)

Courses
- course_id (Primary Key, int, auto-increment)
- tutor_id (int, Foreign Key → users.user_id)
- subject_id (int, Foreign Key → subjects.subject_id)
- course_title (varchar(200), name of the course)
- description (text, detailed overview of the course)
- syllabus (varchar(255), optional file path or URL to syllabus)
- difficulty_level (enum: 'beginner', 'intermediate', 'advanced')
- price (decimal(10,2), course fee in USD)
- start_date (date, start of the course)
- end_date (date, end of the course)
- schedule (text, details of session times or weekly plan)
- max_students (int, default 10, maximum number of allowed students)
- current_students (int, default 0, number of currently enrolled students)
- status (enum: 'upcoming', 'ongoing', 'completed', 'cancelled')
- course_materials (text, optional list or links to course resources)
- location (enum: 'online', 'on_campus', 'off_campus')
- created_at (timestamp, default CURRENT_TIMESTAMP)

Sessions
- session_id (Primary Key, int, auto-increment)
- course_id (int, Foreign Key → courses.course_id, optional link to a course)
- tutor_id (int, Foreign Key → users.user_id)

- student_id (int, Foreign Key → users.user_id)
- subject_id (int, Foreign Key → subjects.subject_id)
- start_time (datetime, scheduled session start)
- end_time (datetime, scheduled session end)
- duration_minutes (int, stored generated column for total session length)
- status (enum: 'scheduled', 'completed', 'cancelled', 'no_show')
- session_type (enum: 'one_on_one', 'group')
- session_link (varchar(255), optional online meeting link)
- session_notes (text, optional notes entered by the tutor or admin)
- student_feedback (text, optional feedback from the student)
- recording_url (varchar(255), optional link to recorded session)
- location (varchar(255), optional physical or virtual location)
- created_at (timestamp, default CURRENT_TIMESTAMP)

Bookings
- booking_id (Primary Key, int, auto-increment)
- student_id (int, Foreign Key → users.user_id)
- tutor_id (int, Foreign Key → users.user_id)
- subject_id (int, Foreign Key → subjects.subject_id)
- requested_date (date, not null)
- requested_time (time, not null)
- duration_requested (int, not null)
- urgency (enum('normal','urgent'), default 'normal')
- status (enum('pending','confirmed','declined','cancelled'), not null)
- message_to_tutor (varchar(500), nullable)
- tutor_response (text, nullable)
- alternative_times (text, nullable)
- created_at (timestamp, default CURRENT_TIMESTAMP)
- updated_at (timestamp, default CURRENT_TIMESTAMP on update CURRENT_TIMESTAMP)

Enrollments
- enrollment_id (Primary Key, int, auto-increment)
- course_id (int, Foreign Key → courses.course_id)
- student_id (int, Foreign Key → users.user_id)
- enrollment_date (timestamp, default CURRENT_TIMESTAMP)
- progress_percentage (int, default 0, tracks student's course progress)
- completion_status (enum: 'enrolled', 'in_progress', 'completed', 'dropped')
- completion_date (timestamp, optional, set when a course is completed)
- certificate_issued (tinyint(1), default 0, indicates if a completion certificate was granted)

Reviews
- review_id (Primary Key, int, auto-increment)
- student_id (int, Foreign Key → users.user_id)
- tutor_id (int, Foreign Key → users.user_id)

- course_id (int, Foreign Key → courses.course_id)
- session_id (int, Foreign Key → sessions.session_id)
- rating (int, 1–5 scale)
- comment (varchar(1000))
- date_posted (timestamp, default CURRENT_TIMESTAMP)
- is_verified (tinyint(1), default 0)
- is_visible (tinyint(1), default 1)
- helpful_count (int, default 0)

Messages

- message_id (Primary Key, int, auto-increment)
- sender_id (int, Foreign Key → users.user_id)
- receiver_id (int, Foreign Key → users.user_id)
- conversation_id (varchar(100), groups messages in the same chat)
- message_text (text, message content)
- attachment_url (varchar(255), optional file or image link)
- timestamp (timestamp, default CURRENT_TIMESTAMP)
- is_read (tinyint(1), default 0, marks if message has been read)
- is_deleted (tinyint(1), default 0, marks if message was deleted by user)

Availability

- availability_id (Primary Key, int, auto-increment)
- tutor_id (int, Foreign Key → users.user_id)
- day_of_week (enum: 'Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday', 'Sunday')
- start_time (time, tutor's available start time)
- end_time (time, tutor's available end time)
- is_recurring (tinyint(1), default 1, marks if availability repeats weekly)
- specific_date (date, optional specific availability date)
- is_booked (tinyint(1), default 0, marks if the slot has been booked)

Notifications

- notification_id (Primary Key, int, auto-increment)
- user_id (int, Foreign Key → users.user_id)
- type (enum: 'booking', 'payment', 'message', 'course', 'session', 'review')
- title (varchar(100), short summary of the notification)
- content (text, detailed message body)
- related_id (int, optional reference to related record such as session_id or review_id)
- priority (enum: 'low', 'normal', 'high', default 'normal')
- timestamp (timestamp, default CURRENT_TIMESTAMP)
- is_read (tinyint(1), default 0, marks if the notification was opened)
- is_email_sent (tinyint(1), default 0, marks if an email notification was also sent)

Professor_verifications

- verification_id (Primary Key, int, auto-increment)
- professor_email (varchar(100), email of the verifying professor)
- professor_name (varchar(100), full name of the verifying professor)
- department (varchar(100), professor's academic department)
- verified_tutor_id (int, Foreign Key → users.user_id)
- verification_date (timestamp, default CURRENT_TIMESTAMP)
- comments (text, optional remarks from the professor)

Campus_resourses

- resource_id (Primary Key, int, auto-increment)
- resource_name (varchar(100), name of the resource)
- resource_type (enum: 'tutoring_center', 'study_room', 'library')
- building (varchar(100), building where the resource is located)
- room_number (varchar(20), optional room identifier)
- hours_of_operation (varchar(255), operating hours or schedule)
- contact_info (varchar(255), optional phone number or email)
- is_available (tinyint(1), default 1, indicates if the resource is open for use)

Media Storage:

Profile pictures or uploads are stored as files in an /uploads folder.
The database only stores the file path or filename (no BLOBs).

Search and Filter:

Search is intentionally simple — users can search by class name only.
Example SQL:
SELECT * FROM courses WHERE course_title LIKE '%math%';

This keeps implementation easy but introduces a non-technical ambiguity issue:
Certain classes share similar names but serve different majors. For example, "Discrete Math" is required for Computer Science majors but not for Mathematics majors.
As a result, a search for "Math" may return both CS-specific and Math-department courses.
This isn't a system error — it's a naming overlap issue. Future versions may fix this by tagging each course with a department or major requirement column.

Algorithms/ Special Processes:

All operations are basic CRUD. No automatic ranking, scheduling, or rating aggregation is implemented at this stage.

Tools and Frameworks:

- Node.js + Express — backend routing
- MySQL + mysql2 — database connection
- bcrypt — password hashing
- HTML/CSS/JS — simple frontend

Summary:

The backend focuses only on essential CRUD functions and class-based search.
It avoids complex automation or logic so development stays lightweight, functional, and easy to expand later.
Known naming overlaps (e.g., “Discrete Math”) are noted but handled manually until department tagging is added.

6. Identify actual key risks for your project at this time

1. Skills Risk – Limited Backend Experience
Some team members are new to Node.js and MySQL, leading to slower development and potential bugs in database queries.
Mitigation: Pair program on core features, follow tutorials/documentation, and divide tasks by strength (front-end vs back-end). Conduct code reviews to share knowledge.

2. Schedule Risk – Tight Timeline for Feature Completion
The team has limited time to implement authentication, tutor search, and scheduling before the deadline.
Mitigation: Prioritize MVP features first (login, search, booking), defer non-essential UI improvements to later iterations, and track progress weekly using a task board (Trello or GitHub Projects).

3. Technical Risk – Server and Database Integration Issues
Unfamiliarity with connecting Node.js to MySQL and handling async queries could cause errors or crashes.

Mitigation: Test database operations in small modules, use connection pooling and error handling, and create mock data for local testing before deployment.

4. Teamwork Risk – Uneven Workload Distribution
Some members are contributing more to coding while others focus mainly on design or documentation.
Mitigation: Assign clear weekly deliverables per member and hold short sync-ups to balance workload and encourage cross-learning.

5. Legal/Content Risk – Use of External Libraries and Assets
Potential use of icons, CSS frameworks, or images without proper licenses.
Mitigation: Use only open-source MIT / Apache licensed packages and royalty-free assets (Unsplash, FontAwesome, Bootstrap CDN) with credit when required.

7. Use of GenAI tools

The use of AI was primarily in creating this document and making sure grammar and other wordings were done correctly and in an efficient manner. The only AI used was chatGPT.

8.Team Lead Checklist

Checklist Item:	Status:
So far all team members are fully engaged and attending team sessions when required.	DONE
Team ready and able to use the chosen back and front end frameworks and those who need to learn are working on learning and practicing	DONE
Team reviewed suggested resources before drafting Milestone 2	DONE
Team lead checked Milestone 2 document for quality, completeness, formatting and compliance with instructions before the submission	DONE
Team lead ensured that all team members read the final Milestone 2 document and agree/understand it before submission	DONE
Team shared and discussed experience with GenAI tools among themselves	DONE

M3 Feedback Summary Report:

Main Search Page

Fill unused space with meaningful content such as tutor spotlights featuring real users to improve engagement and credibility. Update or remove the dropdown filter icon, as most users do not filter directly from the homepage. Fix cursor placement in the search bar so it appears on the correct side of the magnifying glass icon. Adjust yellow color usage to improve visual consistency and readability.

Available Sessions / Tutor Listing Page

Correct logic so unavailable tutors are not displayed as available. Update tutor ratings to remove fractional stars and empty star placeholders. Simplify availability by showing available time slots instead of specific dates to avoid duplicate tutor entries. Fix the session booking feature, which is currently non-functional. Merge the Experience and About sections into a single tutor profile section. Ensure all tutors are registered using an SFSU email address.

Messaging

Fix the broken messaging link so users can reliably communicate. Define and implement a clear way to create new session-related message threads.

Backend

Implement a one-time passcode verification system. Store one passcode per user in the database and associate it with the user's email for secure verification.

Milestone 3 Focus Areasote

Ensure frontend functionality is complete, particularly the dashboard and search features. Fix login authentication issues to ensure reliable access for registered users.

M4

Product Summary

Product Name: GatorTutors

GatorTutors is a tutoring platform built exclusively for San Francisco State University students, connecting learners with verified peer tutors through a secure web interface that streamlines booking, communication, and session tracking. Students can search for tutors by course number, subject, or availability to instantly book on-line or in-person sessions while tutors can manage schedules and review their sessions with ease. Unlike generic services, our product integrates directly with SFSU's course catalog, ensuring matches at the class-code level so students can receive guidance from peers who have already taken the same classes and understand professor expectations. By combining flexibility and familiarity, GatorTutors empowers students to learn from one another, expands academic support beyond traditional tutoring centers, and creates opportunities for peer mentorship.

Major Functions:

- Tutoring site works exclusively for SFSU students
- Students can create an account using their student email
- Students can login to their account
- Students can search for tutors using optional filters
- Students can book tutoring sessions with an available or specific tutor
- Students can communicate with tutors through messages
- Students can view booked tutoring sessions in their dashboard
- Students can view their inbox in their dashboard
- Students can view their profile in their dashboard
- Students can apply to become a tutor
- Tutors can login to their account
- Tutors can view booked sessions in their dashboard
- Tutors can set their availability in their dashboard

URL: http://18.219.122.227

Usability test plan

1. **Test objectives**
   The primary functionality we are testing is whether students can successfully book a session with an available tutor and view that session on their dashboard. This feature is critical because it represents the core purpose of the platform. By verifying that the booking process works correctly and quickly, we ensure that students can rely on the system sessions without confusion or errors. Testing the dashboard display is equally important as it provides students with a clear record of their upcoming commitments and helps them manage their time effectively.

2. **Test background and setup**
   a. System setup, starting point, what HW tester needs to have: Users can browse available tutors without needing an account, but booking a session requires logging in or signing up with their credentials. After logging in, students can search for a tutor and simply click the “Book Session” button on a tutors session. Once the booking is comfirmed, the platform will automatically redirect users to their dashboard where they will be able to view all booked sessions in one place.
   b. Who are the intended users: The intended users are exclusively SFSU students and peer tutors verified through their official SFSU emails. Students can search for tutors based on course criteria while peer tutors can manage their availability. By restricting access to the campus community, the platform ensures a safe, trusted environment.
   c. URL of the system to be tested: http://18.219.122.227/search.html
   d. Test environment: The test can be conducted anywhere as long as a user has access to the internet. No cameras will be in use to capture user interactions and there will be no monitoring. Testers need no prior training to complete the booking sessions and the only constraint is that they have a registered account.

3. **Usability Task description**
   a. Open the GatorTutors website in a browser.

b. If you do not already have an account, sign up using your SFSU email. If you already have one, login.
c. Use the search bar to locate an available tutor.
d. On the tutors available session, click the “Book Session” button to schedule a session.
e. Confirm that the system redirects you to your dashboard.
f. Verify that the booked session appears with correct details.

4. Plan for Evaluation of effectiveness

a. Define clear success criteria (session booked and visible on dashboard)
b. Recruit representative SFSU students to perform the booking task
c. Track task completion rates to see how many succeed without errors
d. Not any user errors, confusion, or repeated attempts
e. Collect user feedback on ease of use and satisfaction
f. Compare results against expected outcomes to identify gaps

5. Plant for Evaluation of efficiency

a. Record the time it takes for users to complete the booking and dashboard viewing task
b. Count the number of clicks or steps required to finish the process
c. Track whether users can complete the task on their first attempt
d. Note any uneccessary actions or detours taken during the task
e. Compare average completion time across multiple participants
f. Identify points where users hesitate or slow down
g. Evaluate if system responses occur quickly
h. Assess whether the dashboard updates instantly after booking

6. Plan for Evaluation of user satisfaction

Please take a moment to answer the following questions.	**Strongly Agree**	**Agree**	**Neutral**	**Disagree**	**Strongly Disagree**
1. It was easy for me to book a tutoring session and view it on my dashboard	○	○	○	○	○
2. The interface was clear and I understood what to do at each step	○	○	○	○	○

3. I am satisfied with the overall experience of using the platform to book and track sessions	○	○	○	○	○

7. **GenAI use**
    a. Microsoft Copilot, GPT-5
    ChatGPT, GPT-5.2
    Google Gemini, Gemini 2.0
    b. GenAI was leveraged to support the usability test plan development process in two ways. First, it was used as an ideation tool to help generate and refine tests that could be used on the product, helping the team explore a wider range of options. Second, it assisted in debugging and improving code by providing explanations and optimizations when the testing failed.
    c. For example, the team needed to design usability tests for the student dashboard. The following prompt was used to come up with ideas to how it should be tested: “Suggest usability test cases for a student dashboard on a tutoring website that includes upcoming sessions, an inbox, and student profile.”. Another example involved debugging code to display the user's profile even when the database fields were empty instead of returning an error. We used this prompt to explain the error and suggest a solution: “Explain why this SQL update is failing when no fields are passed and suggest a fix to this code:...”
    d. Microsoft Copilot - HIGH
    Google Gemini - LOW

QA Test Plan & QA Testing

1.**Test Objectives**

The objective of this QA test plan is to verify that students can successfully book sessions with tutors, with the session appearing on their dashboard, and the system updates components correctly.

2. **HW and SW setup:**

Hardware Setup:

- Laptop or desktop computer with a standard keyboard and mouse/trackpad

Software Setup:

- Operating system: Windows 10/ Windows 11
- Web Browsers Used for Testing:
  - Google Chrome
  - Mozilla Firefox:
- Stable internet connection

Application URL:

- http://18.219.122.227

3. **Feature to be tested.**

The features under test for the booking session include: Searching for available tutors, booking a session through the “Book Session” button, automatic redirection to the student dashboard, display of the booked session with correct details, and removal of the session from available sessions list.

4. **QA test plan: in table format:**

Test #	Test title	Test Description	Test input	Expected output	Test results
1	Search for available tutor	Verify that the search bar returns available tutors	Enter a course code (e.g. MATH).	A list of available tutors with at least one session visible.	
2	Book a tutoring session	Test that the student can successfully book	Click “Book Session” on the tutors	A pop up message should confirm the session was booked	

		a session	session	and the page should redirect to user dashboard	
3	Verify the student dashboard	Ensure the booked session appears on the dashboard	After booking, view the student dashboard	The booked session should appear with the correct information	

5. Testing on two major platforms:
   - Google Chrome - PASS
   - Firefox - PASS

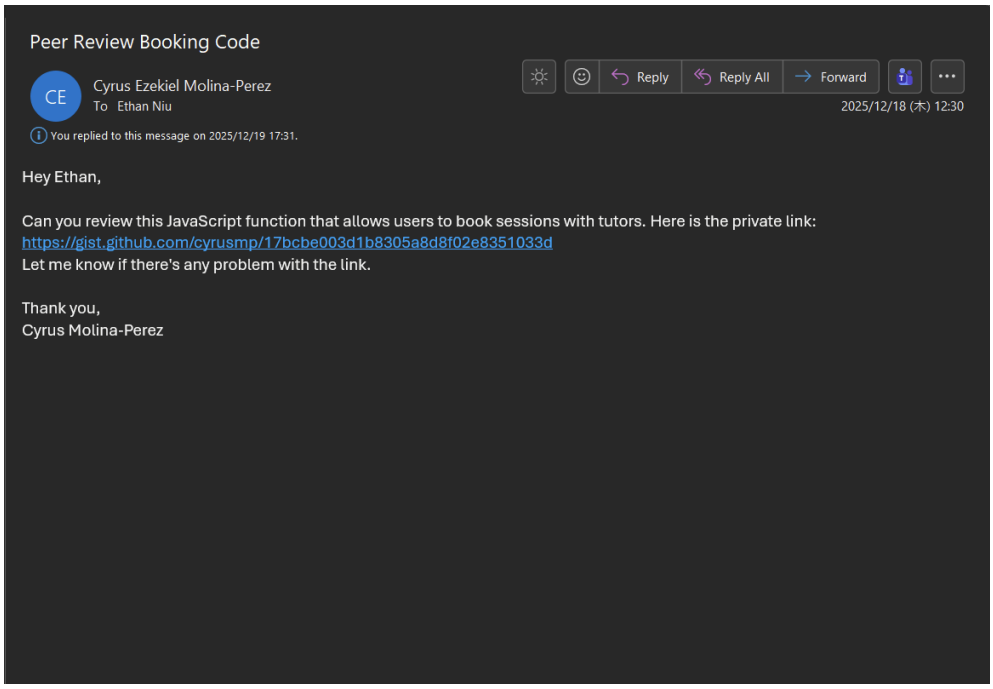
6. GenAI use:
   - **What GenAI tool and version:**
     ChatGPT model GPT-5.2

   - **Explain briefly how you used the tool and what benefit it offered.**
     It provided a clear explanation of what the key functional differences are between QA Testing and the usability test plan. ChatGPT was used to give an example of what a test plan should look like given the Milestone 4 requirements.

   - **Provide brief examples of key examples and prompts**
     ChatGPT was given the milestone 4 requirements and was asked if it could help with creating a QA test plan, the given example was used as a base for the QA test plan, and modified to fit our website.

   - **RANK utility of GenAI**
     GenAI utility: MEDIUM

Peer Code Review



Peer Review Booking Code

CE Cyrus Ezekiel Molina-Perez
To Ethan Niu

Reply | Reply All | Forward

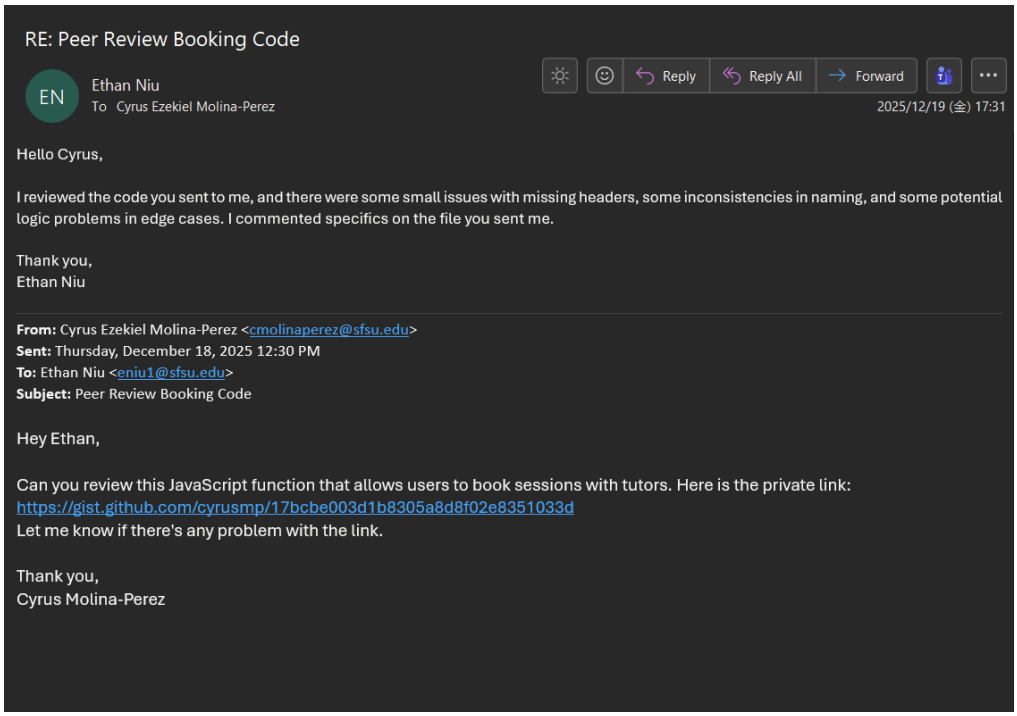
2025/12/18 (木) 12:30

You replied to this message on 2025/12/19 17:31.

Hey Ethan,

Can you review this JavaScript function that allows users to book sessions with tutors. Here is the private link:
https://gist.github.com/cyrusmp/17bcbe003d1b8305a8d8f02e8351033d
Let me know if there's any problem with the link.

Thank you,
Cyrus Molina-Perez



RE: Peer Review Booking Code

EN Ethan Niu
To Cyrus Ezekiel Molina-Perez

Reply | Reply All | Forward

2025/12/19 (金) 17:31

Hello Cyrus,

I reviewed the code you sent to me, and there were some small issues with missing headers, some inconsistencies in naming, and some potential logic problems in edge cases. I commented specifics on the file you sent me.

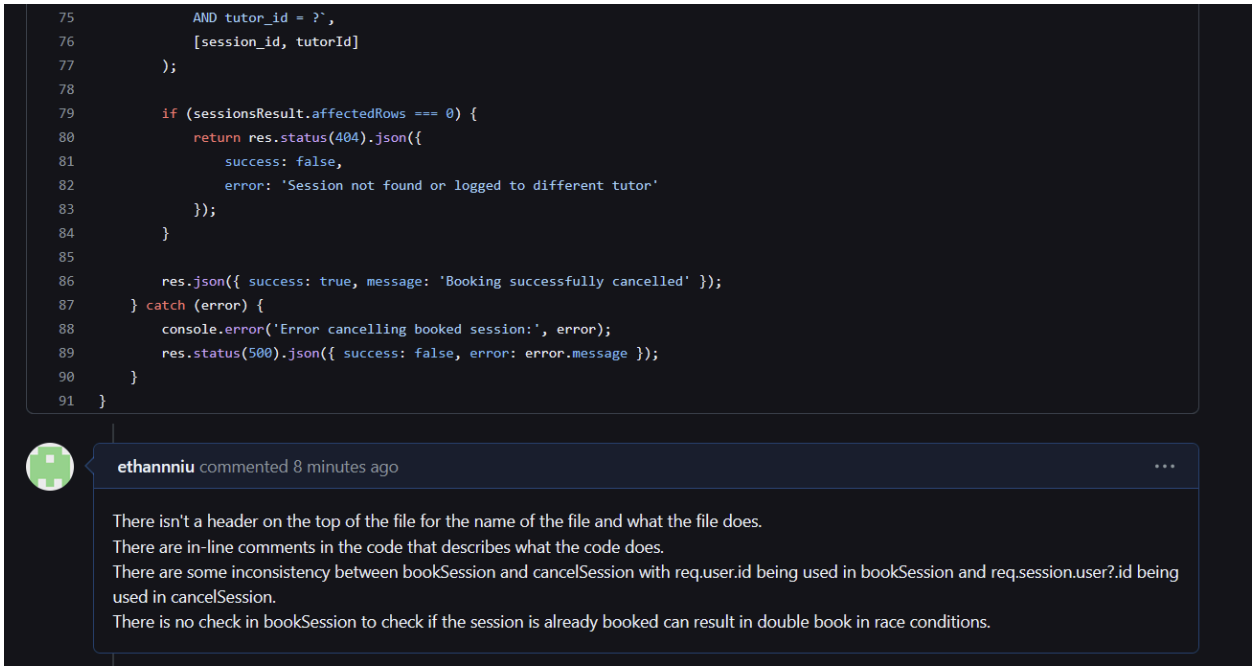
Thank you,
Ethan Niu

From: Cyrus Ezekiel Molina-Perez <cmolinaperez@sfsu.edu>
Sent: Thursday, December 18, 2025 12:30 PM
To: Ethan Niu <eniu1@sfsu.edu>
Subject: Peer Review Booking Code

Hey Ethan,

Can you review this JavaScript function that allows users to book sessions with tutors. Here is the private link:
https://gist.github.com/cyrusmp/17bcbe003d1b8305a8d8f02e8351033d
Let me know if there's any problem with the link.

Thank you,
Cyrus Molina-Perez



```
            AND tutor_id = ?`,
            [session_id, tutorId]
        );

        if (sessionsResult.affectedRows === 0) {
            return res.status(404).json({
                success: false,
                error: 'Session not found or logged to different tutor'
            });
        }

        res.json({ success: true, message: 'Booking successfully cancelled' });
    } catch (error) {
        console.error('Error cancelling booked session:', error);
        res.status(500).json({ success: false, error: error.message });
    }
}
```


GenAI session for code review:

Sure — this is a solid start and the intent is clear. I'll review it in a **backend / security / correctness** lens and then suggest **concrete improvements** with examples.

High-level assessment ✅

What you did well

- Clear separation of concerns (`bookSession` vs `cancelSession`)
- Parameterized queries (✅ protects against SQL injection)
- Proper HTTP status codes in most cases
- Good comments explaining assumptions (auth middleware, etc.)
- Updating `tutoring_sessions.is_booked` consistently

Key issues & improvements 🔴🟡🟢

🔴 1. Inconsistent auth source (`req.user` vs `req.session.user`)

You use **two different auth mechanisms**:

```
// bookSession
const student_id = req.user.id;

// cancelSession
const tutorId = req.session.user?.id;
```

⚠️ This is dangerous and confusing.

Why it matters

- If your app standardizes on JWT / authMiddleware → `req.user`
- Mixing session-based and token-based auth can break access control

✅ **Recommendation**
Use **one** auth source everywhere (likely `req.user`):

```
const tutorId = req.user.id;
```

🔴 2. No transaction (data integrity risk)

Your booking flow is **not atomic**:

```
INSERT INTO bookings ...
UPDATE tutoring_sessions SET is_booked = 1
```

If the second query fails:

- Booking exists
- Session still appears available

🚨 Same problem in `cancelSession`.

✅ Fix: use a transaction

```
const conn = await pool.getConnection();
try {
  await conn.beginTransaction();

  const [result] = await conn.query(
    `INSERT INTO bookings (student_id, tutor_id, session_id)
     VALUES (?, ?, ?)`,
    [student_id, tutor_id, session_id]
  );

  await conn.query(
    `UPDATE tutoring_sessions
     SET is_booked = 1
     WHERE id = ? AND is_booked = 0`,
    [session_id]
  );

  await conn.commit();
  res.json({ success: true, booking_id: result.insertId });
} catch (err) {
  await conn.rollback();
  throw err;
} finally {
  conn.release();
}
```

🔴 3. Race condition: double booking possible

Two students can book the same session at the same time.

Current logic:

- No check that `is_booked = 0`
- No unique constraint enforcement

✅ Best practice

Update with condition:

```
UPDATE tutoring_sessions
SET is_booked = 1
```

```
WHERE id = ? AND is_booked = 0
```

Then verify:

```
if (updateResult.affectedRows === 0) {
  return res.status(409).json({
    success: false,
    error: "Session already booked"
  });
}
```

🟢 Bonus: add DB-level protection

`UNIQUE (session_id)`

on `bookings`

🟡 4. Missing role validation

You assume:

- `bookSession` → student
- `cancelSession` → tutor

But you never enforce role explicitly.

✅ **Add role checks**

```
if (req.user.role !== 'student') {
  return res.status(403).json({ error: "Students only" });
}

if (req.user.role !== 'tutor') {
  return res.status(403).json({ error: "Tutors only" });
}
```

🟡 5. `session_id` trust issue

In `cancelSession`, the client supplies both:

```
{ booking_id, session_id }
```

⚠️ A malicious client could mismatch them.

✅ **Safer approach**
Derive `session_id` from the booking:

```
DELETE FROM bookings
WHERE booking_id = ? AND tutor_id = ?
RETURNING session_id
```

Then update the returned session.

🟡 6. HTTP semantics

You say:

```
// DELETE /api/book
```

But you still read from `req.body`.

🟡 This is allowed, but REST-wise it's better to:

```
DELETE /api/book/:booking_id
```

And use:

```
req.params.booking_id
```

🟢 7. Error exposure

```
res.status(500).json({ error: err.message });
```

In production, this can leak DB details.

✅ Better:

```
res.status(500).json({
  success: false,
  error: "Internal server error"
});
```

Summary of recommended fixes

Area	Fix
Auth	Use `req.user` consistently
Data integrity	Wrap booking/cancel in transactions
Race conditions	Conditional updates (`is_booked = 0`)
Authorization	Enforce roles (student/tutor)
Security	Don't trust client `session_id`
API design	Improve DELETE route semantics
Errors	Hide internal error messages

Overall verdict ⭐

Functionality: ✅ Good
Security: ⚠️ Needs tightening
Concurrency safety: ❌ Needs transactions
Design clarity: 🟡 Minor inconsistencies

If you want, I can:

- Refactor this into a **production-ready version**
- Add **unit tests**

- Review your **DB schema**
- Align this with **JWT auth middleware**

Just tell me what you want next 👍

What GenAI tool and version you used:

ChatGPT version GPT-5.2

Explain briefly how you used the tool and what benefit it offered.

I asked ChatGPT to help me review the code, and it offered a comprehensive rundown of problems in the code that might have otherwise not have been made known.

Provide brief examples of key examples and prompts.

I pasted the code in the file to ChatGPT and asked “can you review this code?”

RANK utility of GenAI here as LOW or MEDIUM or HIGH

Medium

Self Check on Best Practices for Security

Our team conducted a security review of the tutoring platform to ensure all major assets are protected from any threats. The table below summarizes the key assets, the threat they face, and solutions we've implemented. We confirm that all user passwords stored in the database are encrypted and that our system performs input validation on all user-submitted fields. We also validated that the search bar input is limited to 40 alphanumeric characters, registered emails must end in "@sfsu.edu", and that users explicitly accepted the Terms & Conditions before account creation.

Asset to be protected	**Types of possible/expected attacks**	**Consequence of security breach**	**Strategy to mitigate/protect asset**
Server	Brute-force SSH login attempts, stolen SSH private keys, unauthorized remote access, privilege escalation attacks	Full server takeover, insertion of malicious code, loss of system integrity	Use SSH key-based authentication
Database	SQL injection, credential theft, data scraping, privilege misuse	Exposure of sensitive academic information, tampering with session availability, loss of user trust	Enforce strong DB passwords, restrict DB access
User Accounts	Credential stuffing, phishing attempts, session hijacking via stolen cookies	Unauthorized booking or cancellation of sessions, access to private messages, reputational damage	Require strong passwords

Self Check of the Adherence to Original Non-Functional Specs

The system is accessible via modern web browsers — **DONE**.
The application has been tested and is accessible using modern browsers including Chrome, Firefox, and Edge on desktop systems.

The system is deployed on a publicly accessible server — **DONE**.
The application is deployed on a publicly accessible AWS EC2 instance using a public IP address and can be accessed by instructors for grading and testing.

The system supports multiple concurrent users — **DONE**.
The backend supports multiple user sessions simultaneously and has been exercised by multiple team members without conflicts or failures.

The system provides reasonable response times for user actions — **DONE**.
Page loads, searches, messaging, and dashboard interactions complete within acceptable response times under normal usage conditions.

The system securely stores user authentication credentials — **DONE**.
User passwords are securely hashed and stored in the database following standard security practices.

The system validates user input to prevent malformed or unsafe data — **DONE**.
Input validation is enforced for critical fields including search inputs, registration data, email format, and required form fields.

The system restricts access to authenticated users for protected features — **DONE**.
Sensitive features such as dashboards, messaging, and booking are accessible only to authenticated users.

The system enforces SFSU-specific access constraints — **DONE**.
User registration is restricted to valid SFSU email addresses, ensuring the platform remains exclusive to the SFSU community.

The system provides fault tolerance and basic error handling — **DONE**.
User-facing error messages are displayed for common failure scenarios, and the system recovers gracefully from invalid actions.

The system maintains data consistency across user sessions — **DONE**.
User profiles, messages, bookings, and session data remain consistent across sessions and page reloads.

The system provides a usable and intuitive user interface — **DONE**.
The interface follows consistent navigation patterns and was refined based on usability feedback collected during earlier milestones.

The system separates frontend and backend responsibilities — **DONE**.
Frontend presentation and backend logic are clearly separated through defined APIs and modular code organization.

The system supports maintainable code organization — **DONE**.
Code is structured into logical components with consistent naming conventions to support readability and maintenance.

The system supports future feature extensibility — **DONE**.
The application architecture allows for additional features and enhancements to be added without major refactoring.

The system protects sensitive user data — **DONE**.
Sensitive information such as passwords and verification data is never exposed in client-side code or logs.

The system provides monitoring and usage visibility — **DONE**.
Google Analytics is integrated to provide visibility into site usage and user engagement.

The system remains available during scheduled demo and grading periods — **DONE**.
The deployed application has remained stable and accessible throughout final testing and demo preparation.

The system complies with all course-defined deployment and documentation requirements — **DONE**.

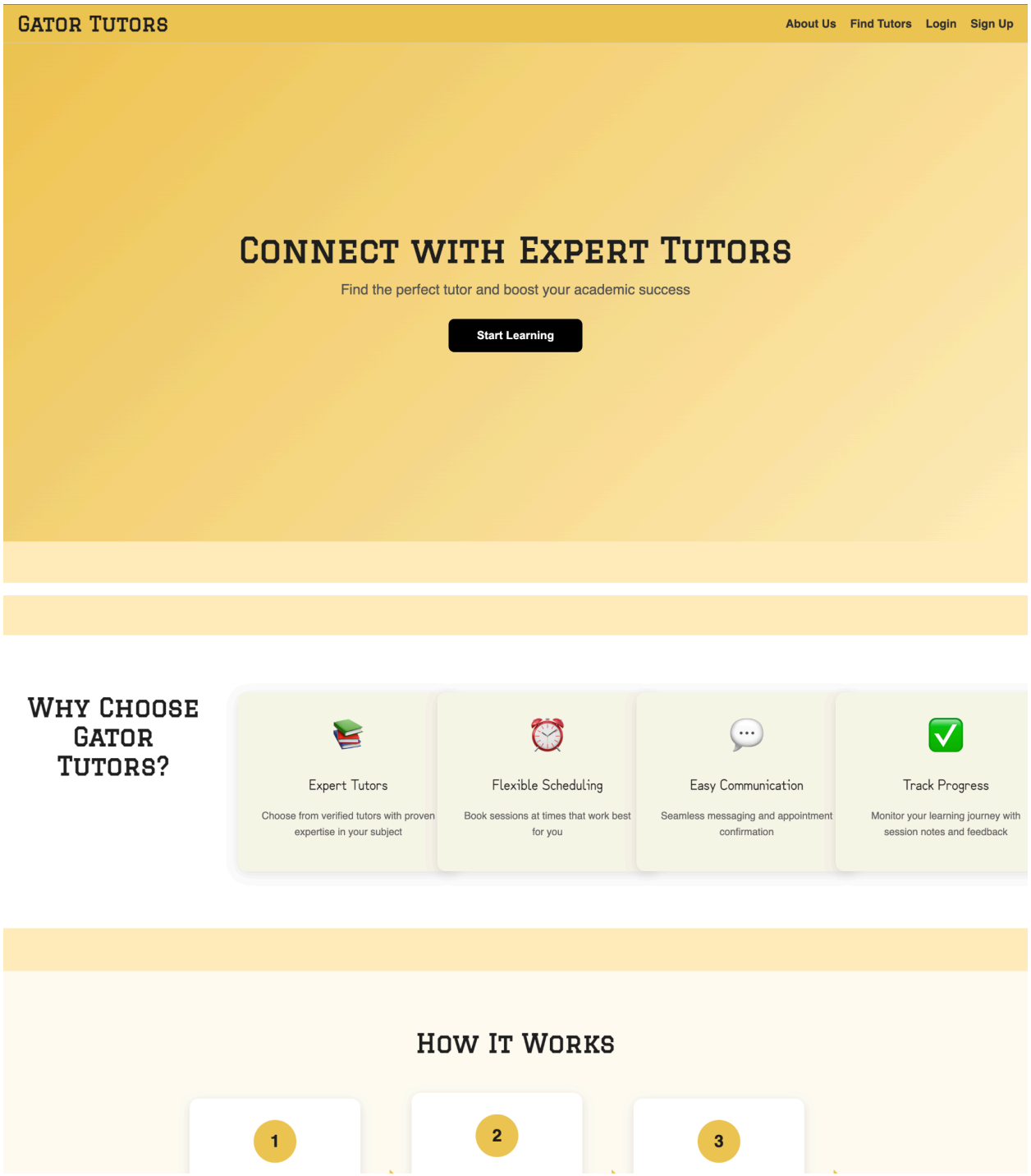
Use of GenAI

GenAI tools were used to support planning, review, and documentation tasks throughout the project rather than to replace core implementation work. ChatGPT was used to help draft and review usability and QA test plans, refine test case coverage, and improve clarity and consistency in project documentation.

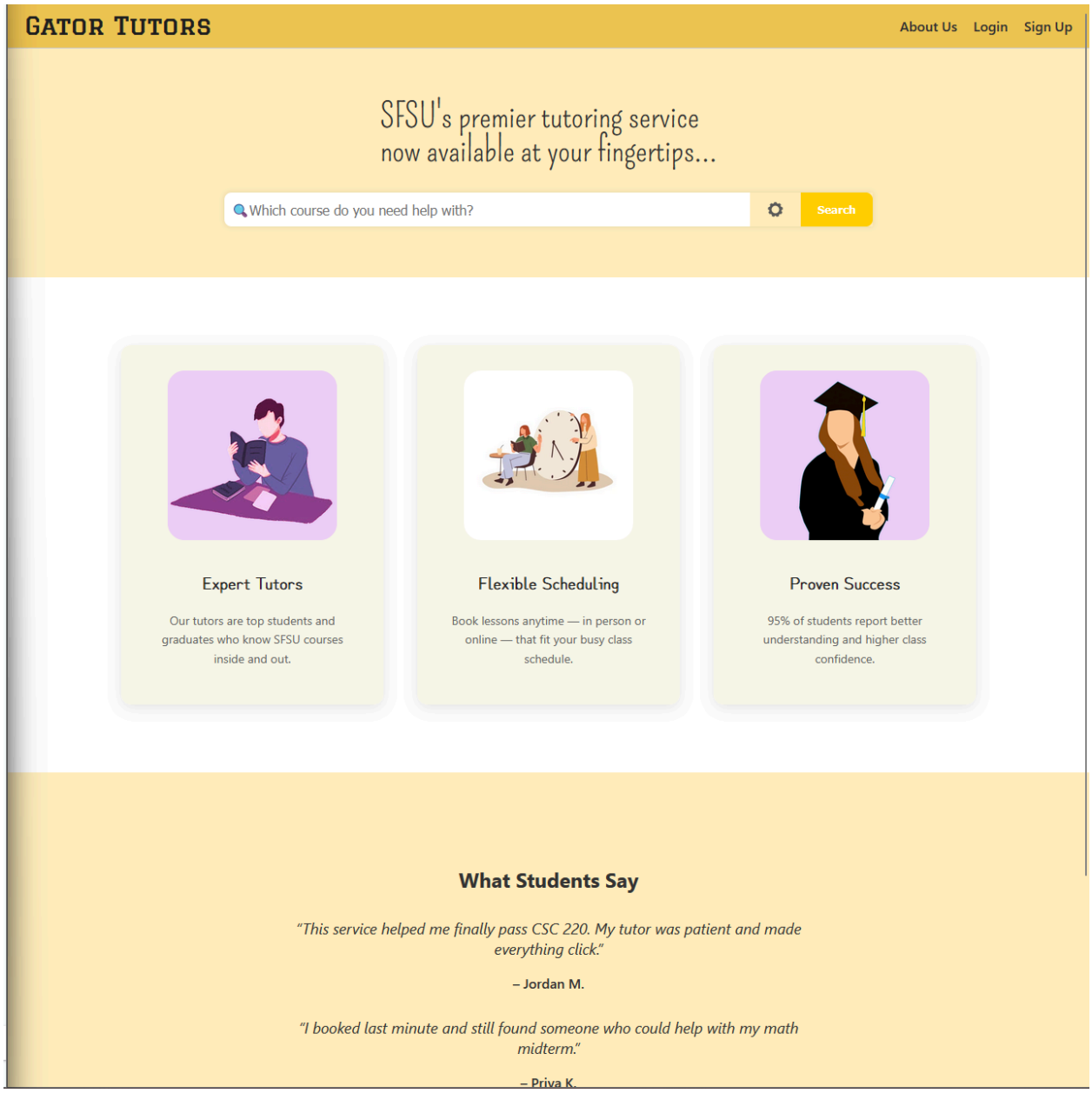
GenAI was also used as a supplementary reviewer during peer code review to identify potential issues related to readability, structure, and best practices. All GenAI suggestions were evaluated by team members, and only accurate and relevant feedback was incorporated.

3. Product Screen Shots

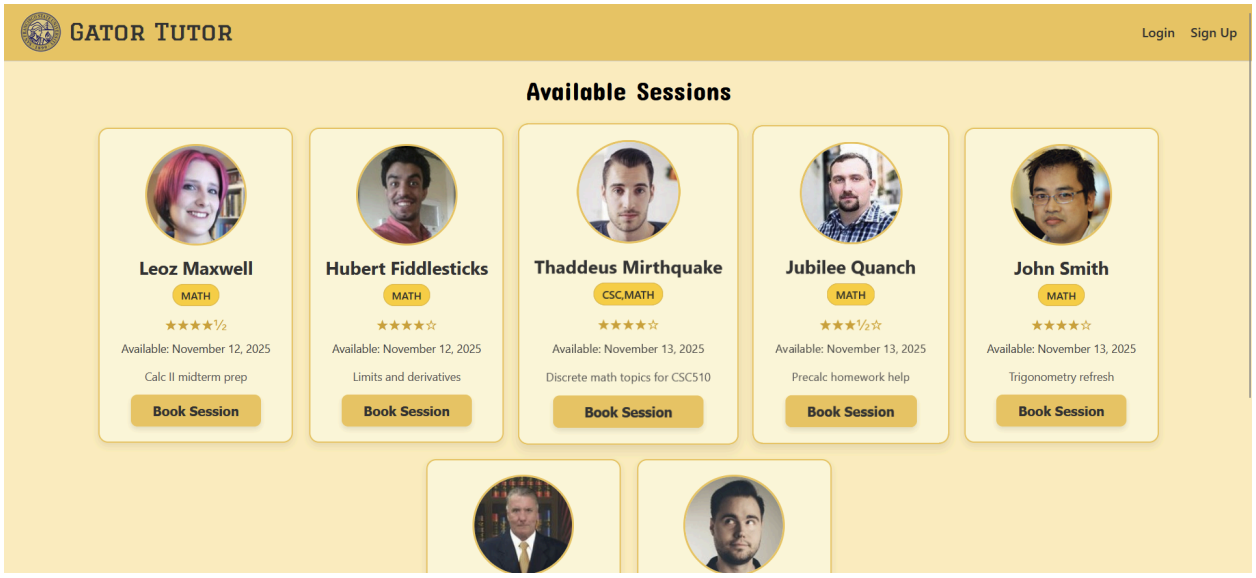
Landing page



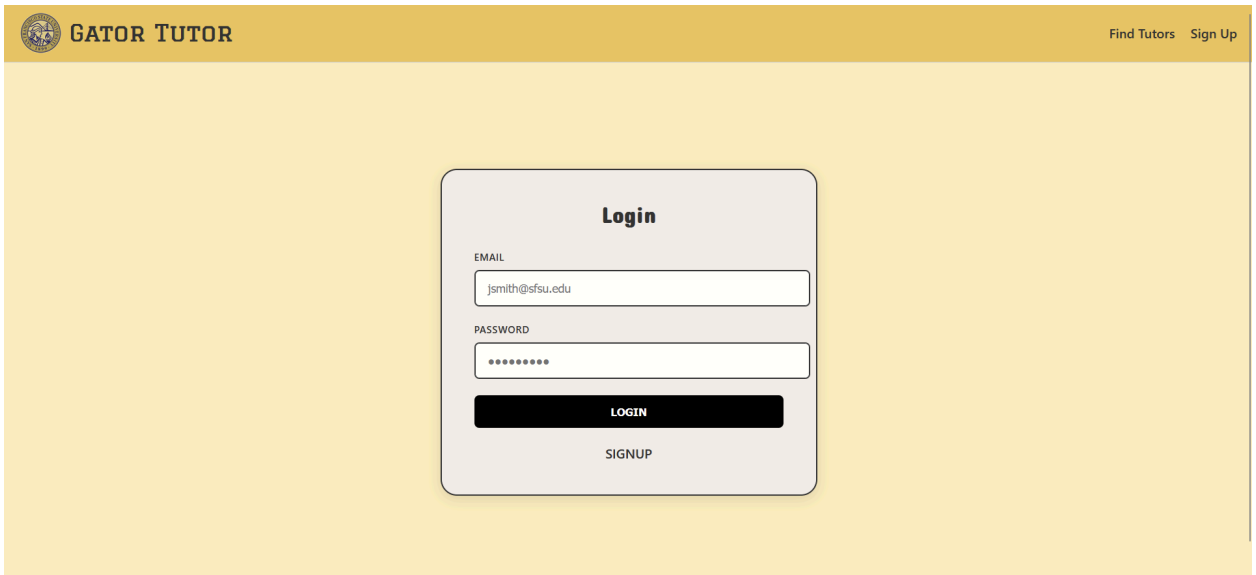
Search page


GATOR TUTORS
About Us
Login
Sign Up
SFSU's premier tutoring service
now available at your fingertips...
Which course do you need help with?
Search
Expert Tutors
Our tutors are top students and graduates who know SFSU courses inside and out.
Flexible Scheduling
Book lessons anytime — in person or online — that fit your busy class schedule.
Proven Success
95% of students report better understanding and higher class confidence.
What Students Say
"This service helped me finally pass CSC 220. My tutor was patient and made everything click."
– Jordan M.
"I booked last minute and still found someone who could help with my math midterm."
– Priya K.

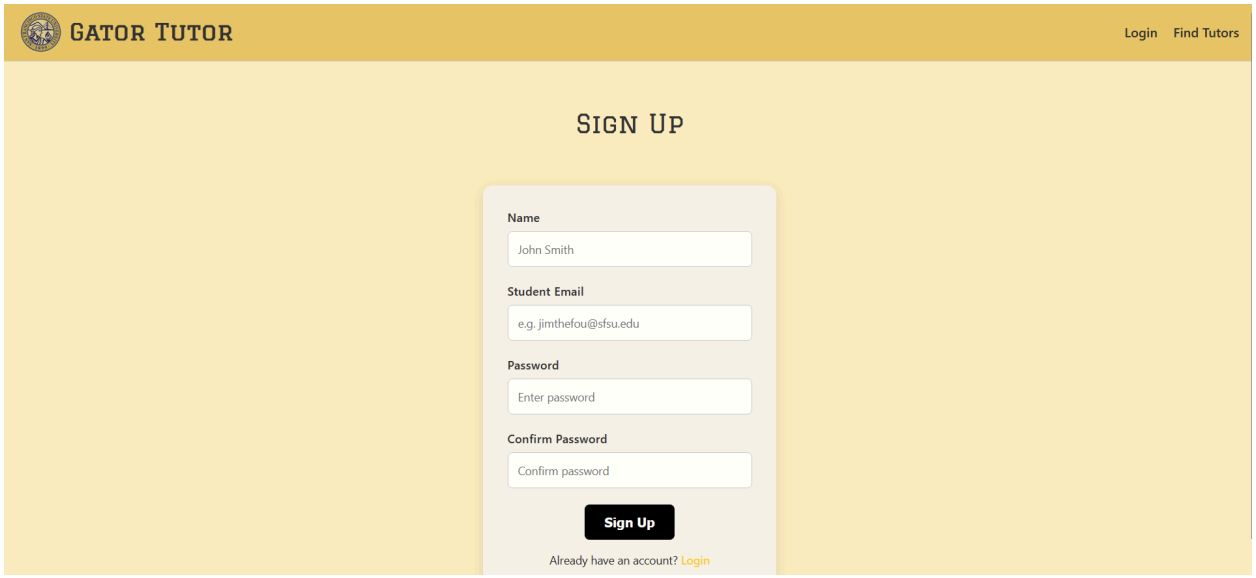
Tutor search page



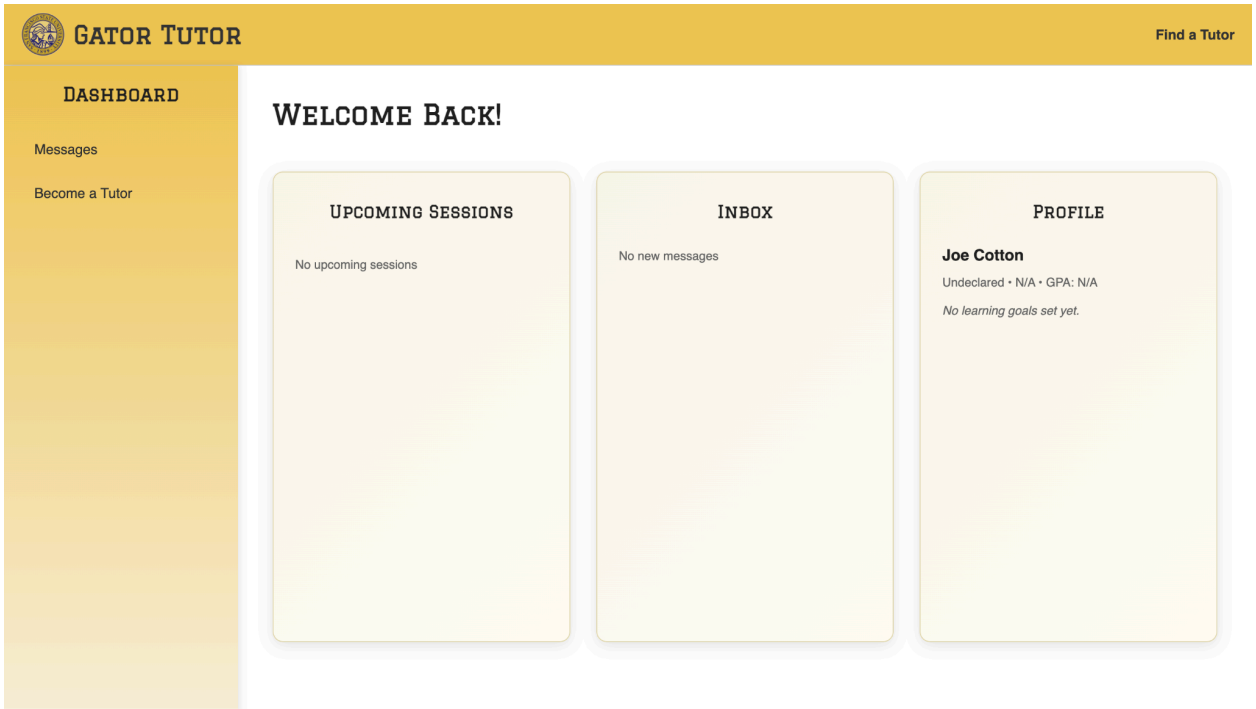
Login page



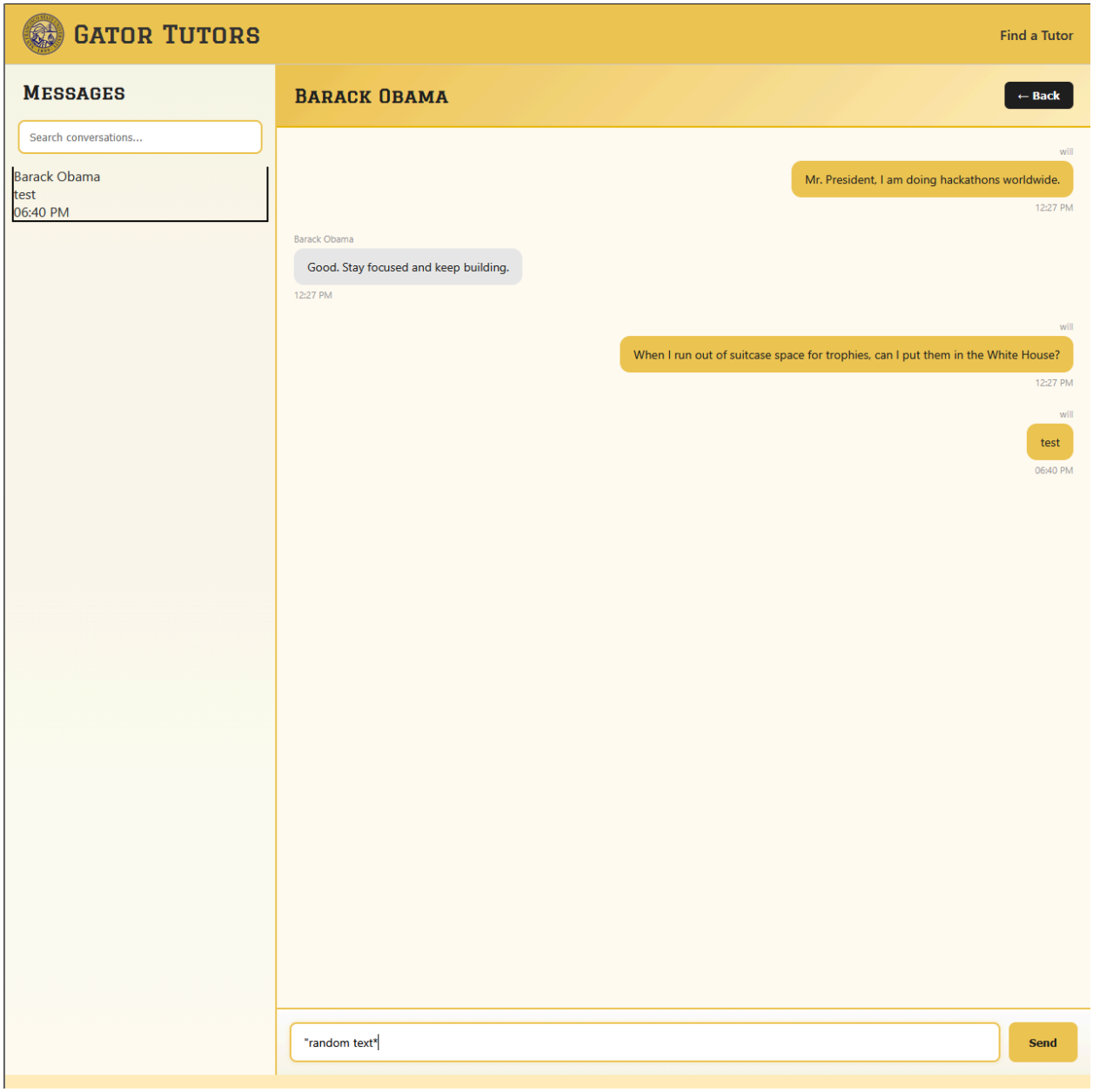
Signup page



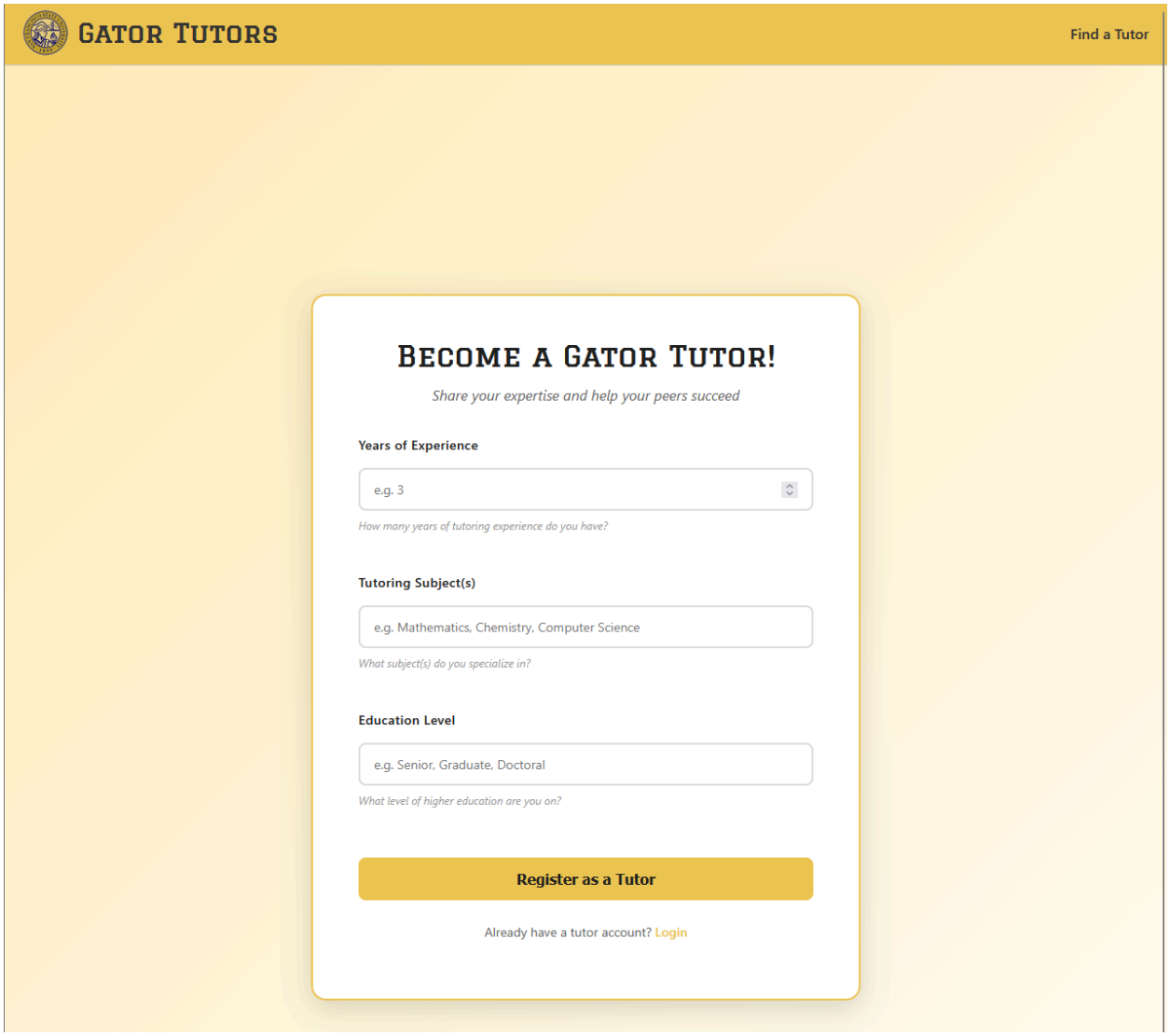
Student dashboard page



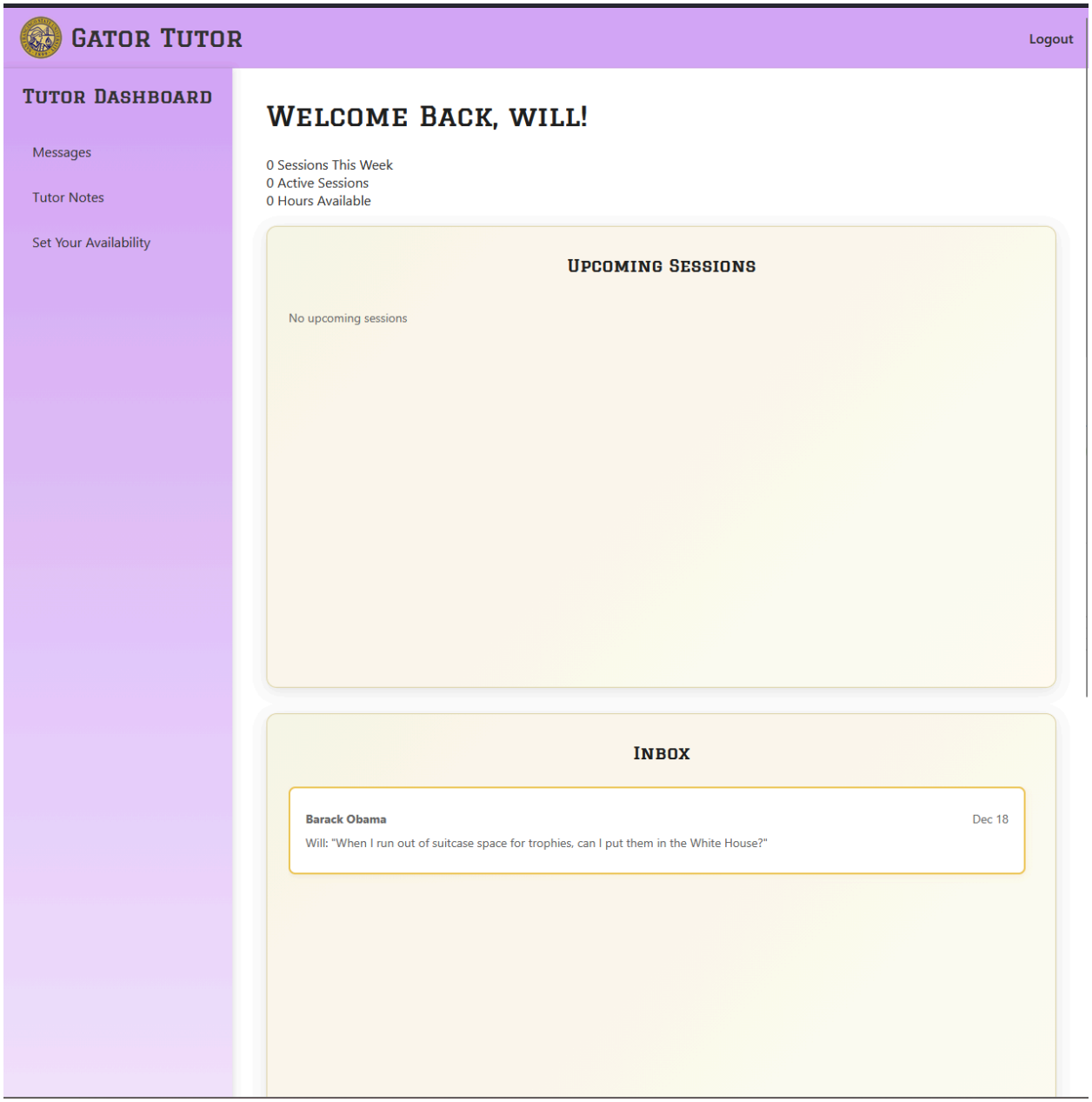
Messaging:


Gator Tutors
Find a Tutor
Messages
Search conversations...
Barack Obama
test
06:40 PM
Barack Obama
← Back
will
Mr. President, I am doing hackathons worldwide.
12:27 PM
Barack Obama
Good. Stay focused and keep building.
12:27 PM
will
When I run out of suitcase space for trophies, can I put them in the White House?
12:27 PM
will
test
06:40 PM
"random text"
Send

Become a tutor:



Tutor dashboard page



Tutor Notes

GATOR TUTOR

Login Sign Up

STUDENT NOTES

Bryce

Allison

Max

Tori

+ Add Student

+ New Note

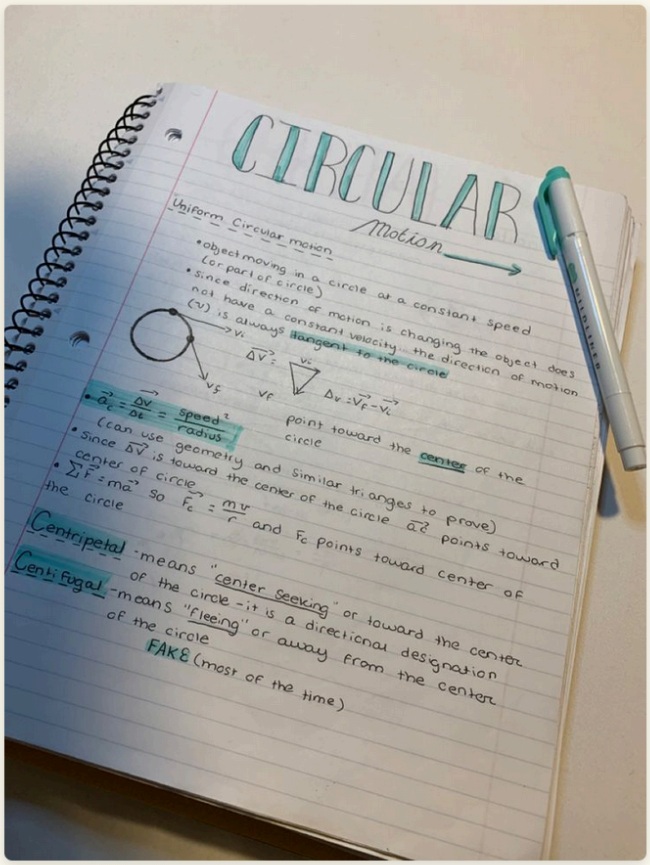
SESSIONS

All Subjects

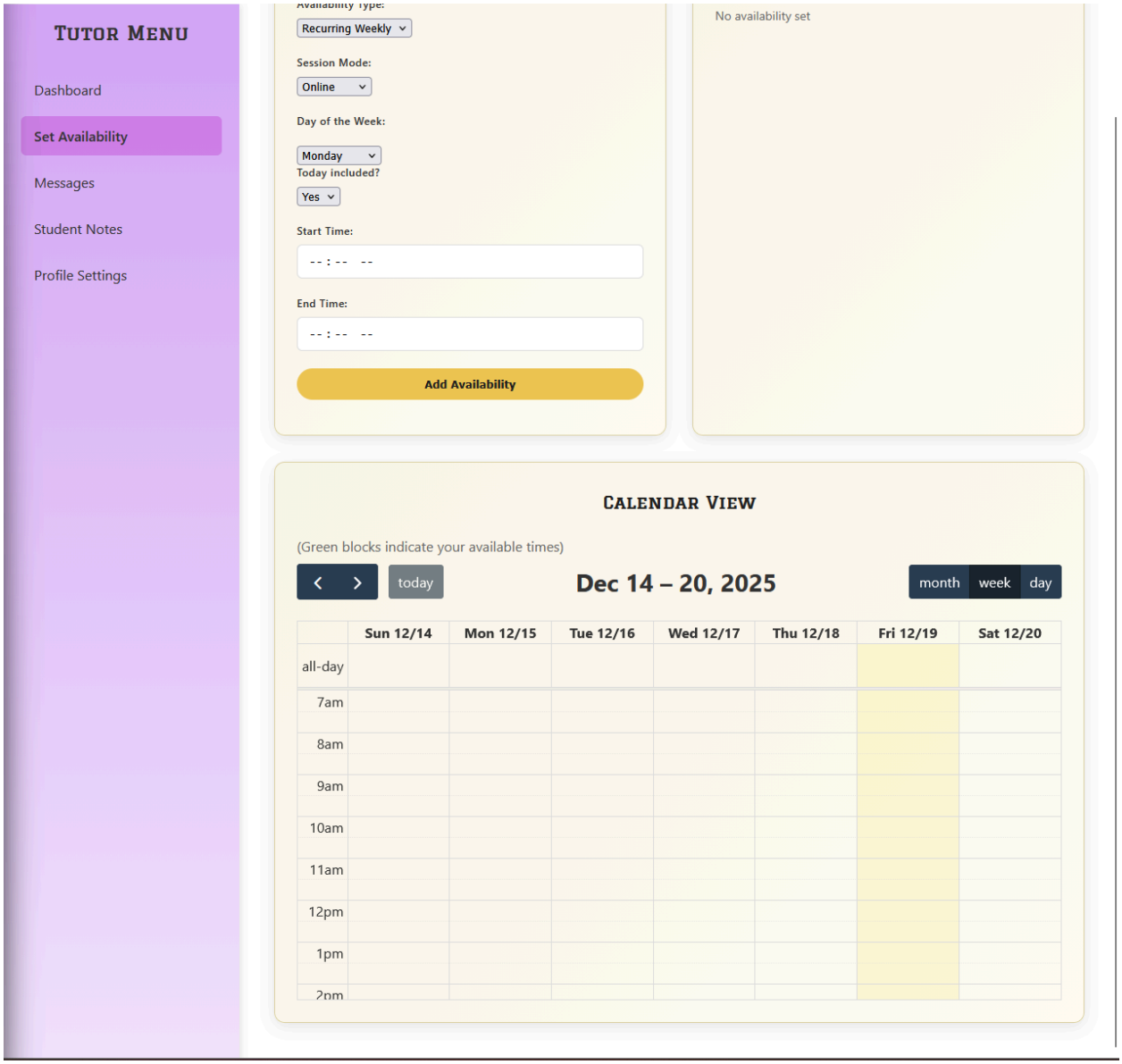
Session 1 - Physics

Session 2 - Data Structures

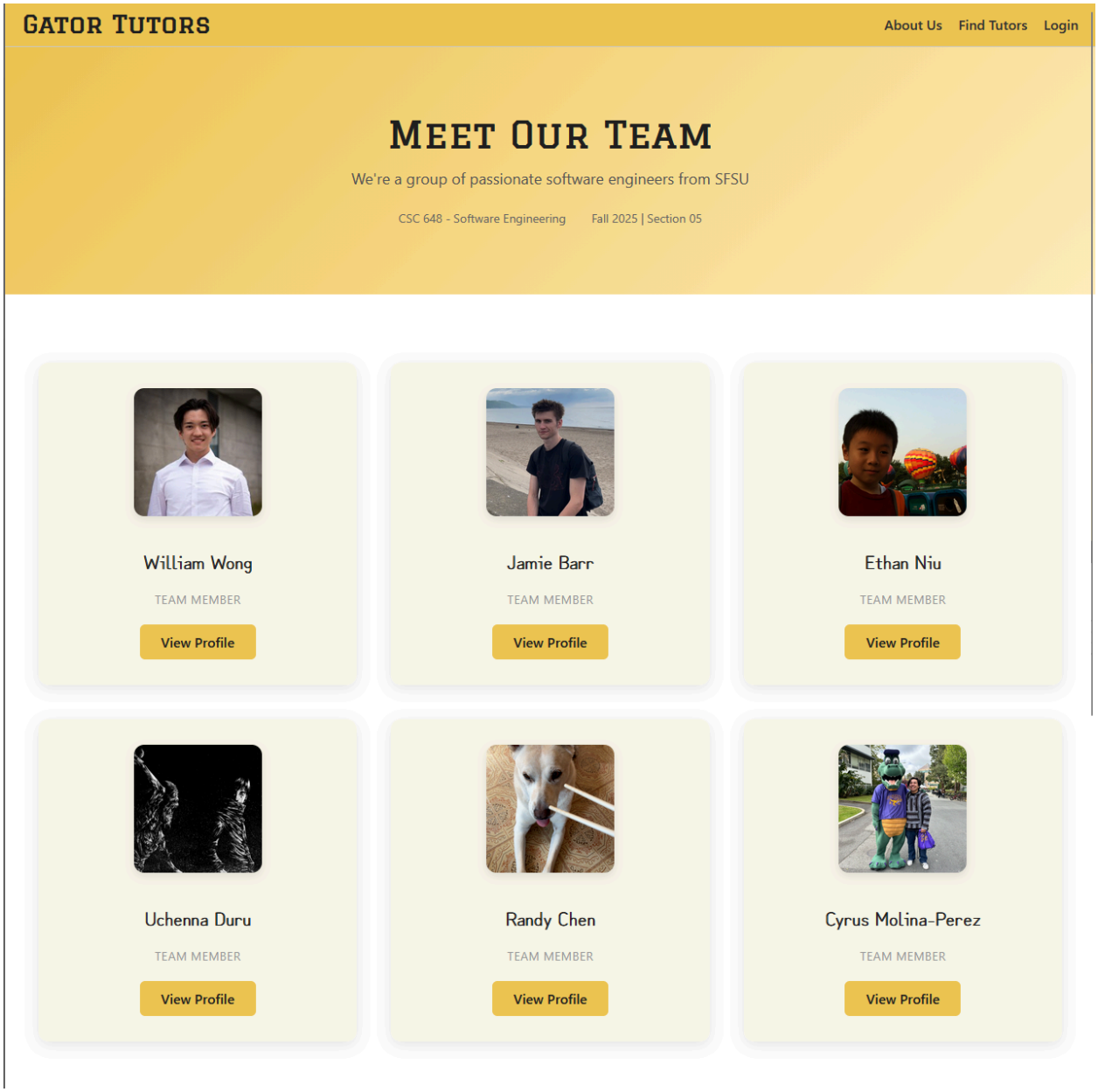
Session 3 - Biology



Tutor Availability

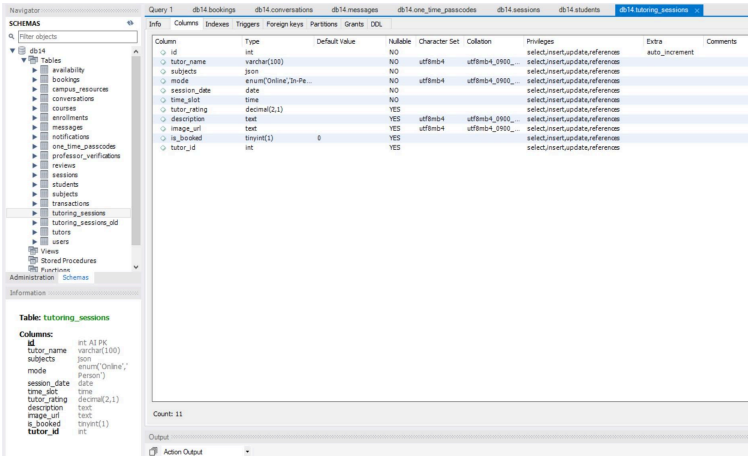


About us:

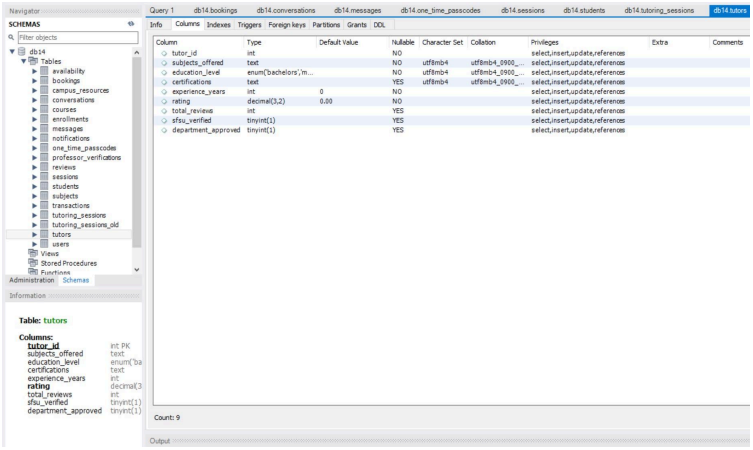

Gator Tutors
About Us
Find Tutors
Login
Meet Our Team
We're a group of passionate software engineers from SFSU
CSC 648 - Software Engineering
Fall 2025 | Section 05
William Wong
TEAM MEMBER
View Profile
Jamie Barr
TEAM MEMBER
View Profile
Ethan Niu
TEAM MEMBER
View Profile
Uchenna Duru
TEAM MEMBER
View Profile
Randy Chen
TEAM MEMBER
View Profile
Cyrus Molina-Perez
TEAM MEMBER
View Profile

4. Database Organization

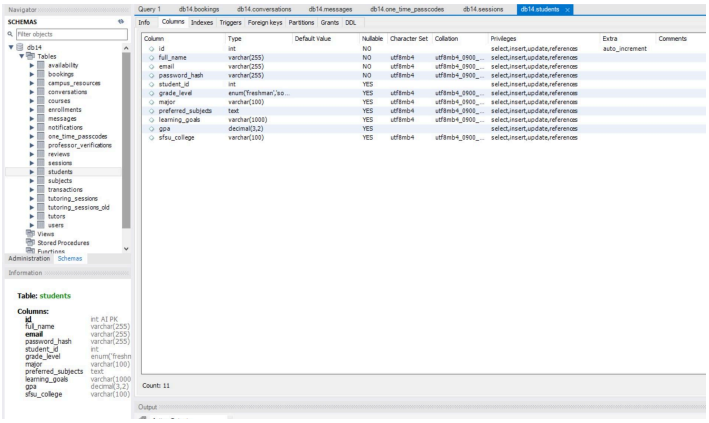
Tutoring Sessions



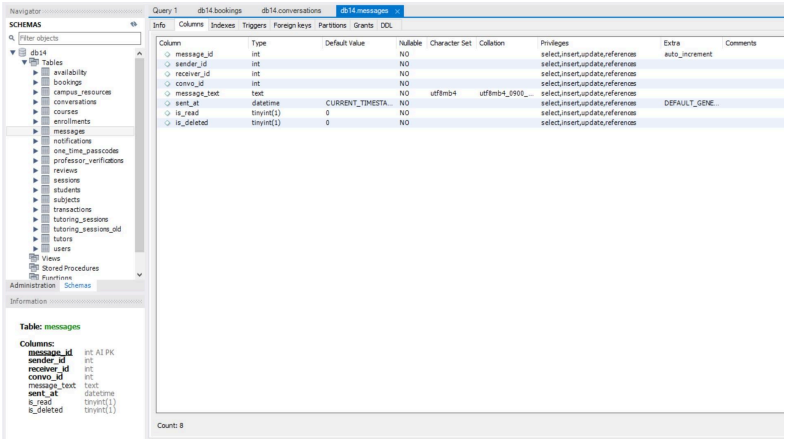
Tutors



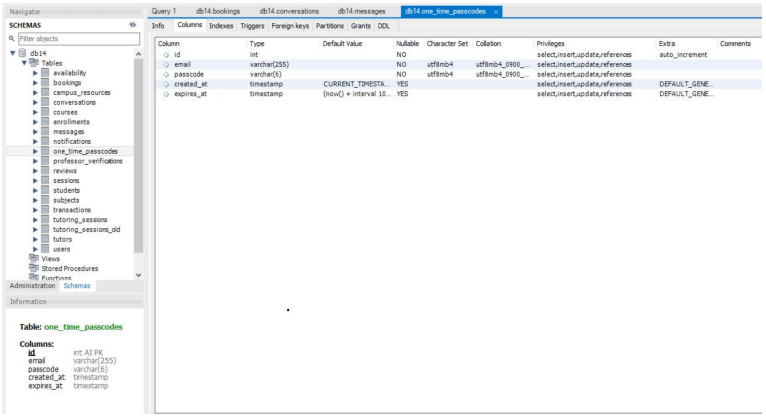
Students



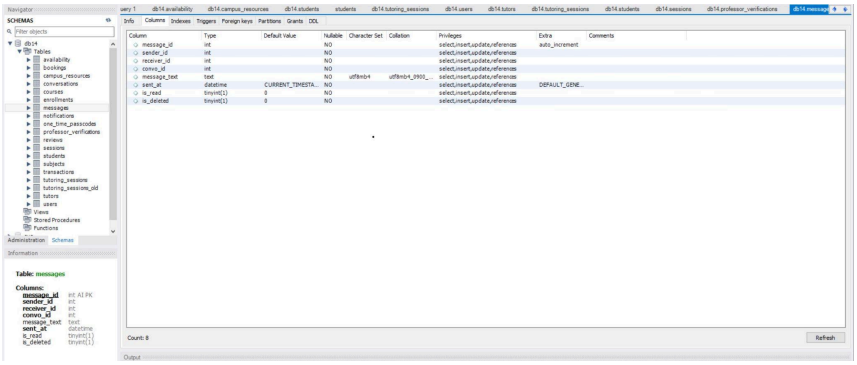
Conversations



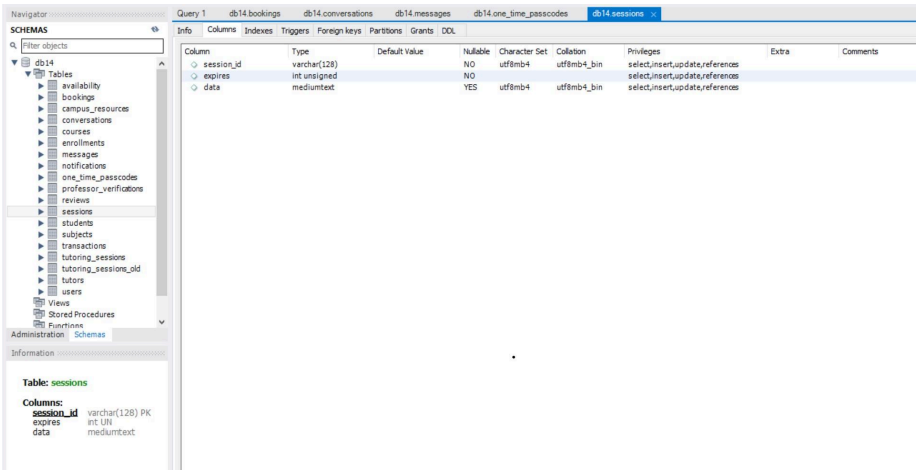
one_time_passcodes



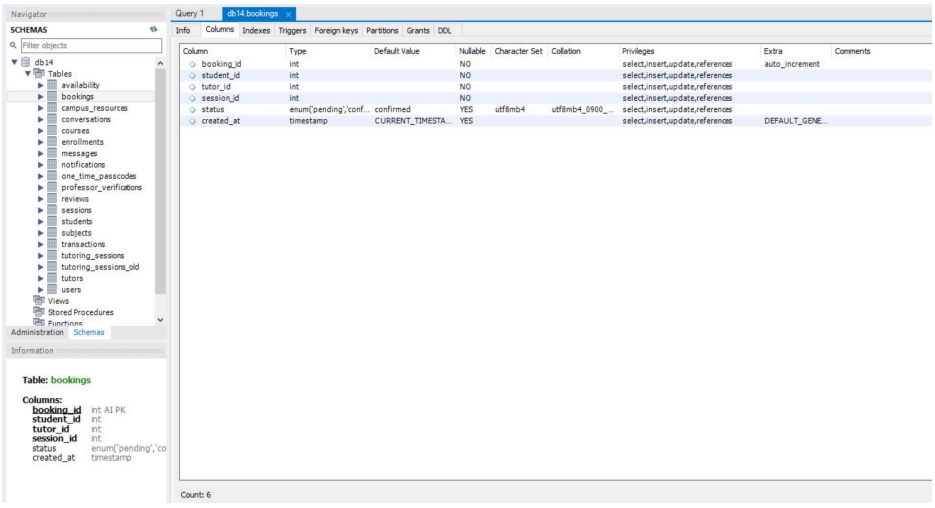
Messages



session


Navigator
SCHEMAS
Filter objects
db14
Tables
availability
bookings
campus_resources
conversations
courses
enrollments
messages
notifications
one_time_passcodes
professor_verifications
reviews
sessions
students
subjects
transactions
tutoring_sessions
tutoring_sessions_old
tutors
users
Views
Stored Procedures
Functions
Administration
Schemas
Information
Table: sessions
Columns:
session_id varchar(128) PK
expires int UN
data mediumtext
Query 1
db14.bookings
db14.conversations
db14.messages
db14.one_time_passcodes
db14.sessions
Info
Columns
Indexes
Triggers
Foreign keys
Partitions
Grants
DDL
Column
Type
Default Value
Nullable
Character Set
Collation
Privileges
Extra
Comments
session_id varchar(128) NO utf8mb4 utf8mb4_bin select,insert,update,references
expires int unsigned NO select,insert,update,references
data mediumtext YES utf8mb4 utf8mb4_bin select,insert,update,references

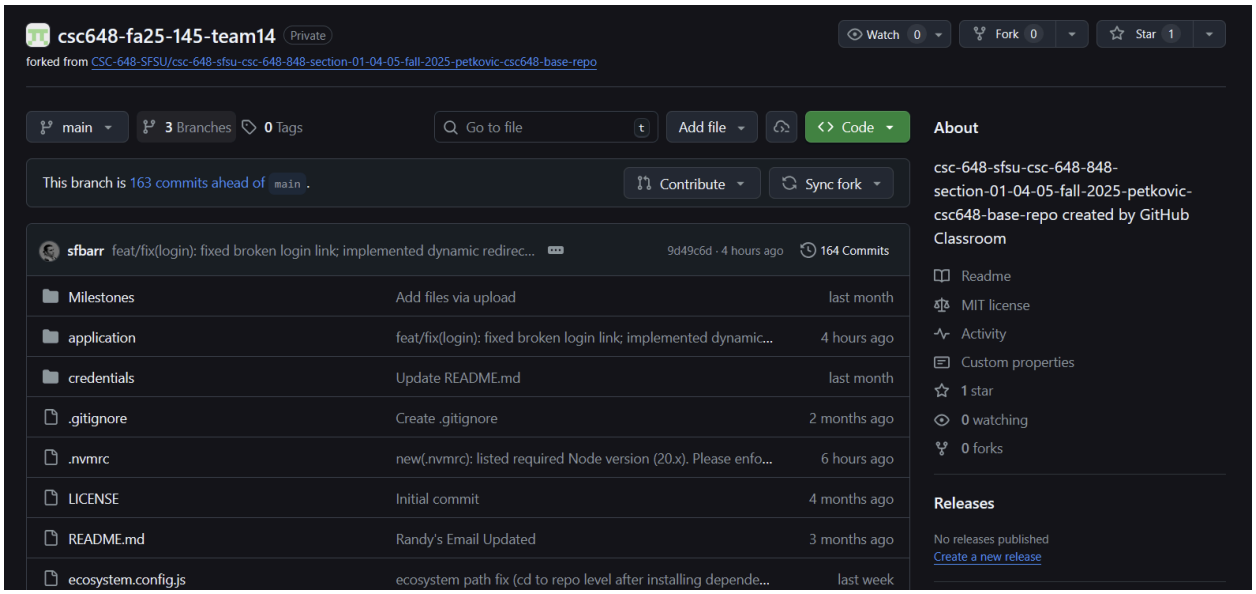
Bookings


Navigator
SCHEMAS
Filter objects
db14
Tables
availability
bookings
campus_resources
conversations
courses
enrollments
messages
notifications
one_time_passcodes
professor_verifications
reviews
sessions
students
subjects
transactions
tutoring_sessions
tutoring_sessions_old
tutors
users
Views
Stored Procedures
Functions
Administration
Schemas
Information
Table: bookings
Columns:
booking_id int AI PK
student_id int
tutor_id int
session_id int
status enum('pending','co
created_at timestamp
Query 1
db14.bookings
Info
Columns
Indexes
Triggers
Foreign keys
Partitions
Grants
DDL
Column
Type
Default Value
Nullable
Character Set
Collation
Privileges
Extra
Comments
booking_id int NO select,insert,update,references auto_increment
student_id int NO select,insert,update,references
tutor_id int NO select,insert,update,references
session_id int NO select,insert,update,references
status enum('pending','conf... confirmed YES utf8mb4 utf8mb4_0900_... select,insert,update,references
created_at timestamp CURRENT_TIMESTA... YES select,insert,update,references DEFAULT_GENE...
Count: 6

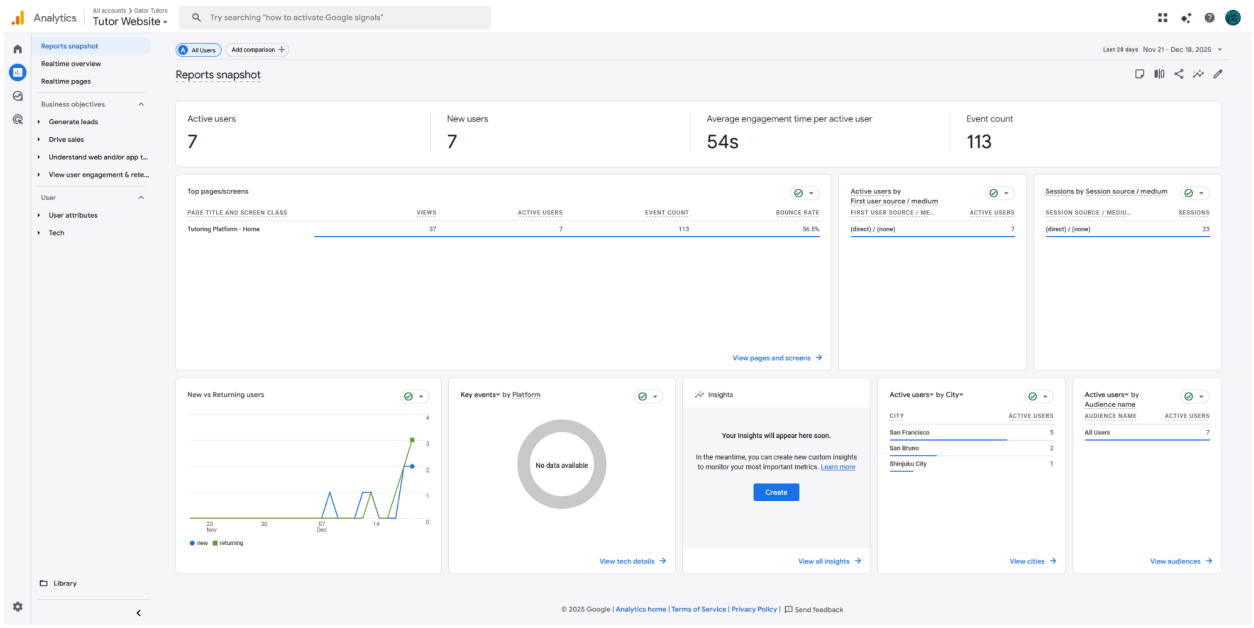
5. GitHub Organization

a. The main branches are main, BackendTeam, and FrontendFixes
b. Everyone on the team had the ability to push to the main/master branch, Jamie and Will were the only authorized team members that could deploy changes to production.

GitHub home page

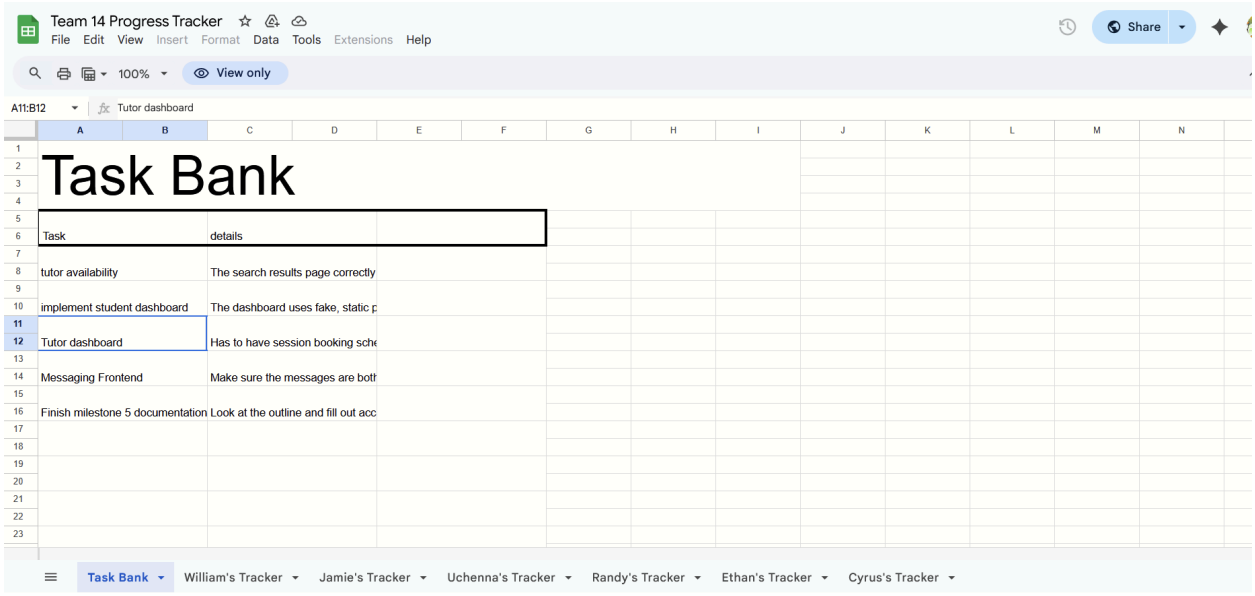


6. Google Analytics Stats Plot

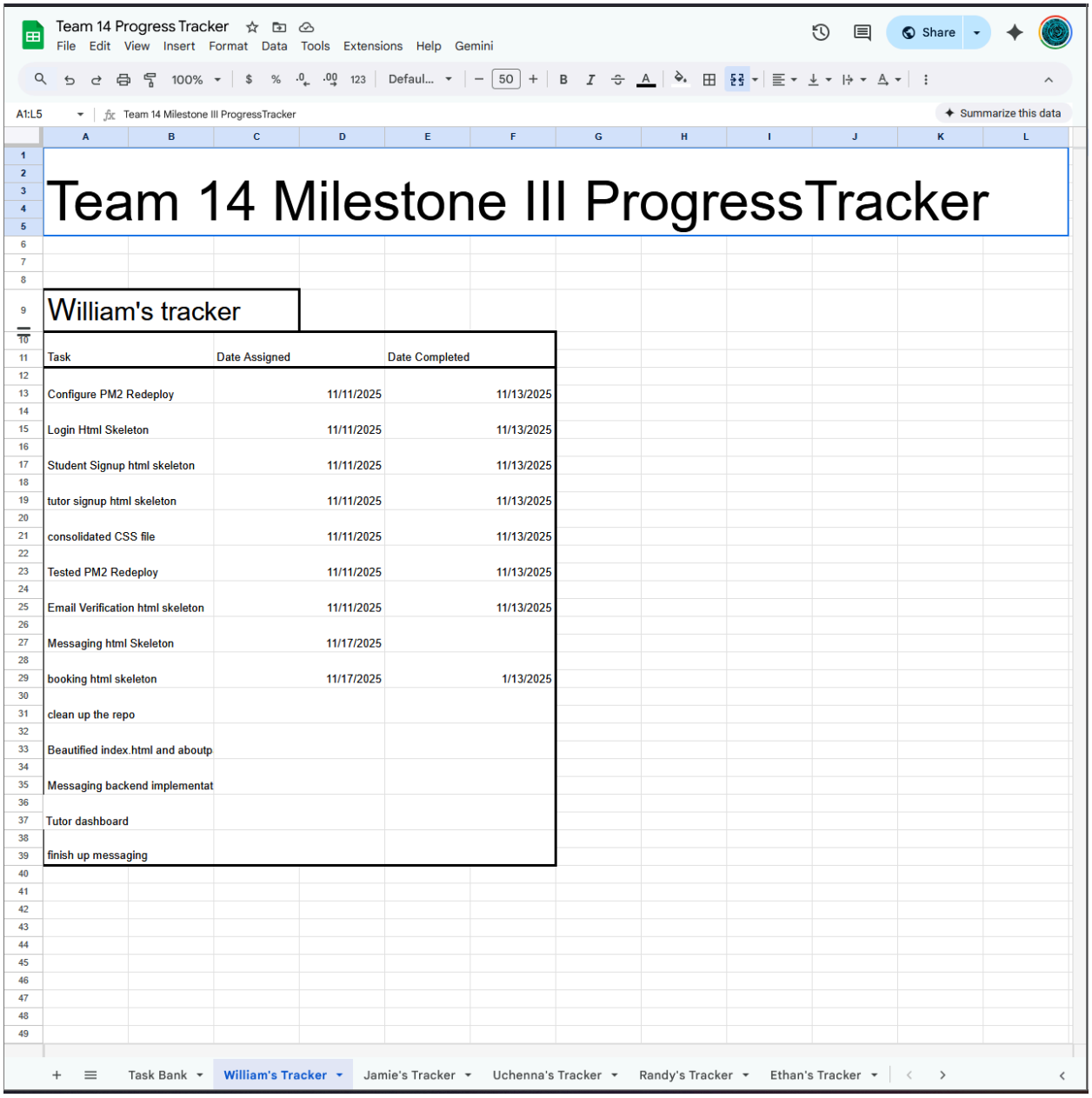


7. Project Management

Google sheets was used to distribute and track tasks that each member needed to complete, and discord was used for communication.

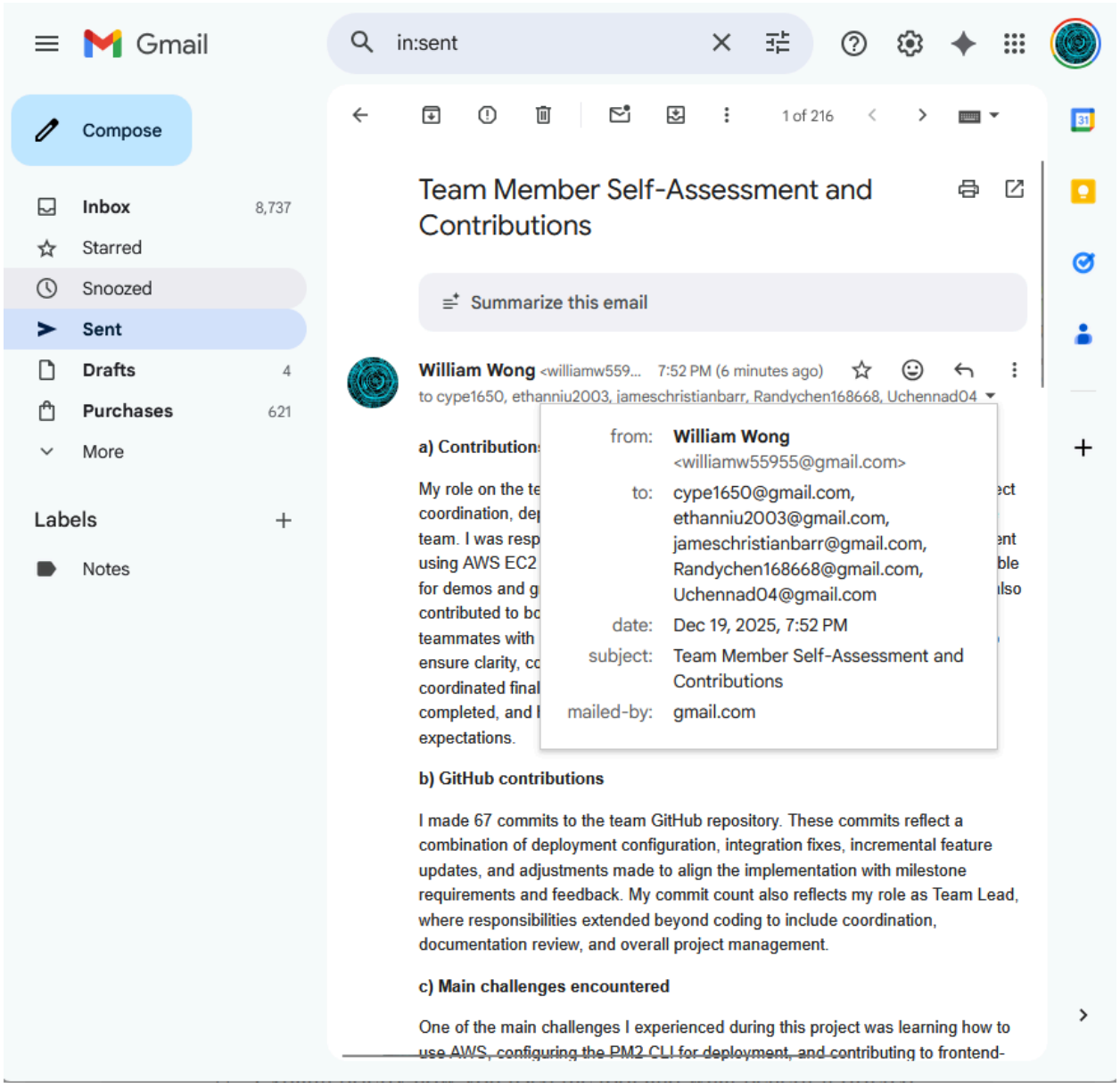


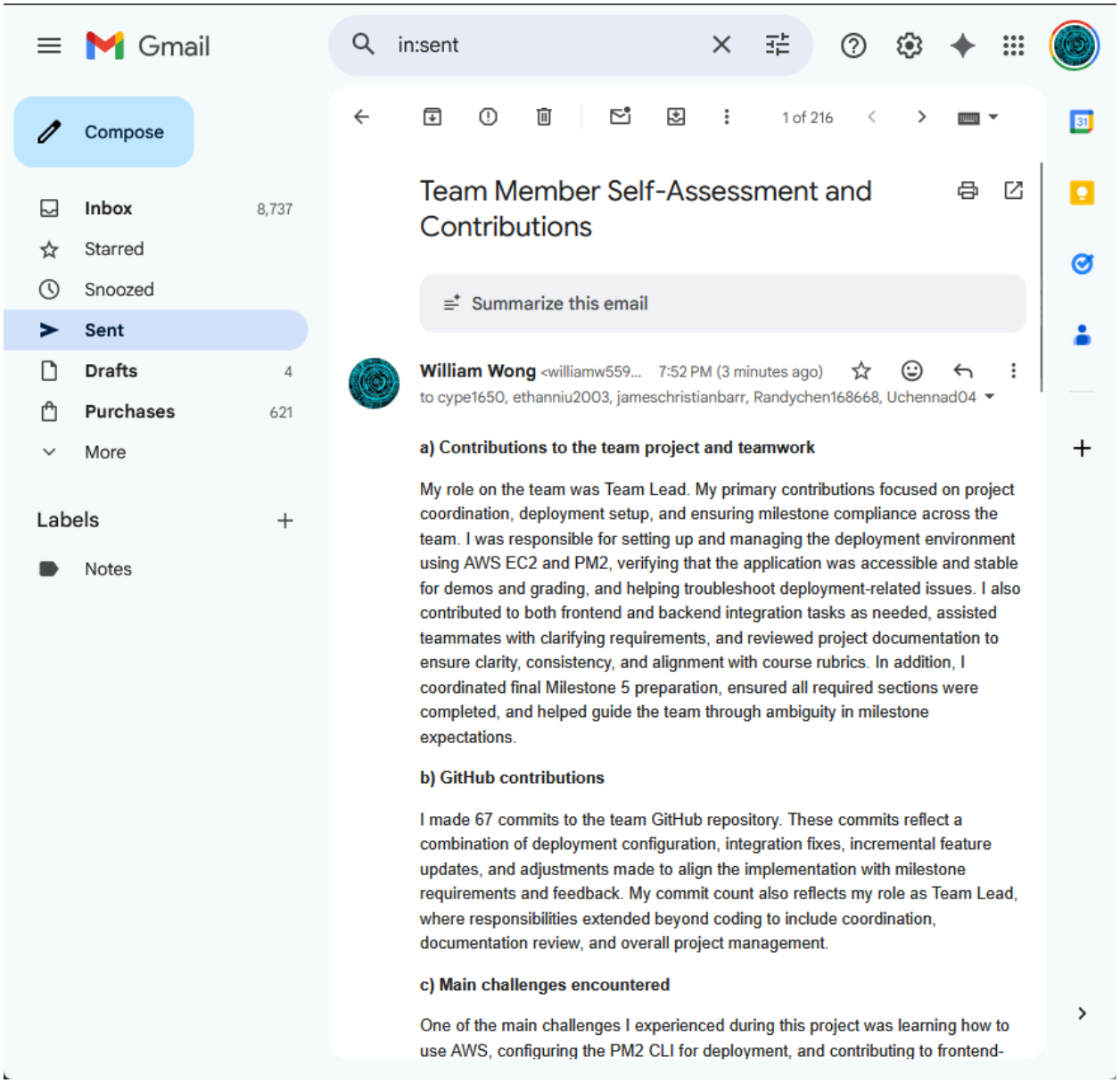
Example individual tracker:

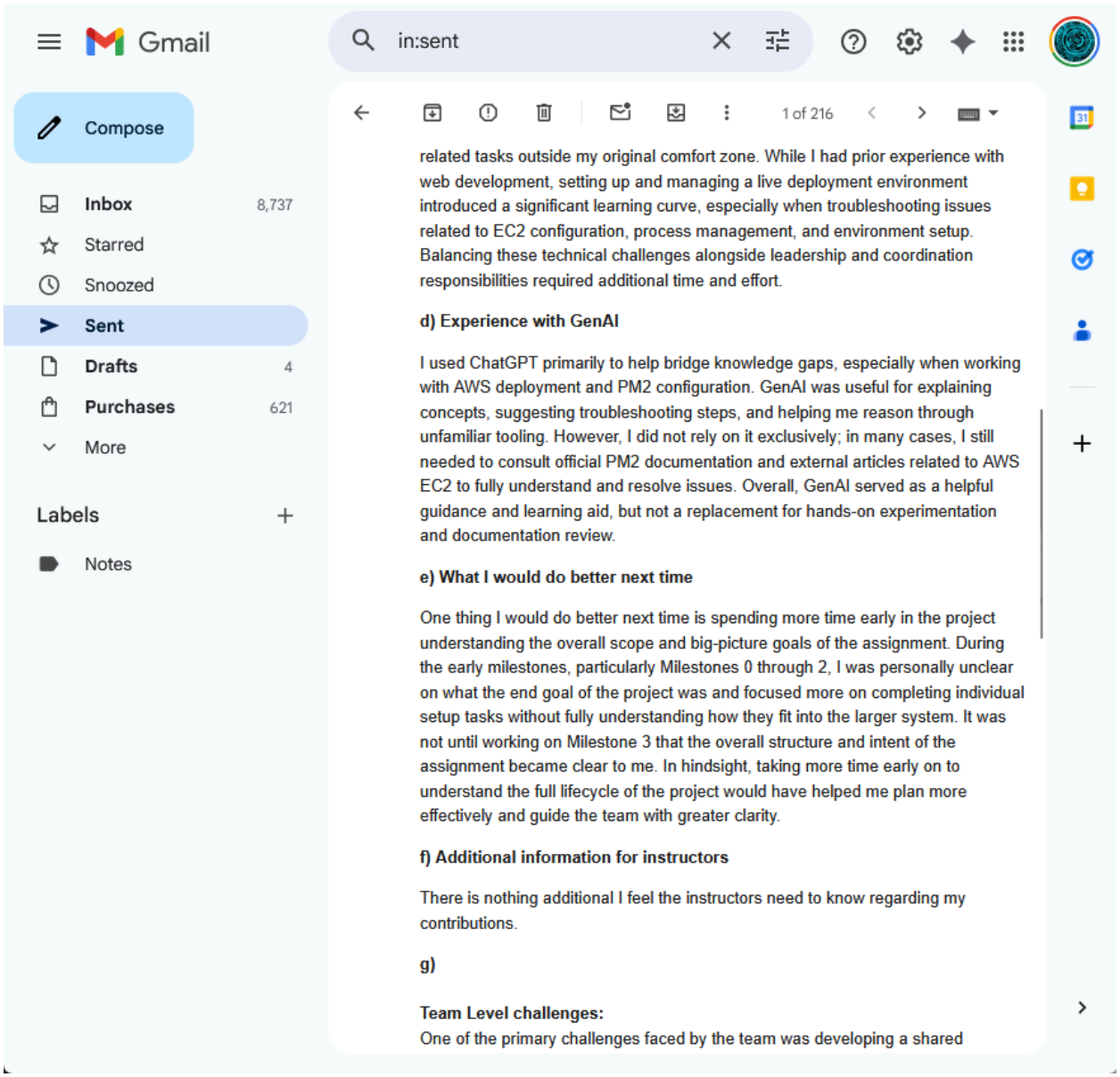

Team 14 Progress Tracker
File Edit View Insert Format Data Tools Extensions Help Gemini
Share
A1:L5
Team 14 Milestone III ProgressTracker
Summarize this data
Team 14 Milestone III ProgressTracker
William's tracker
Task
Date Assigned
Date Completed
Configure PM2 Redeploy 11/11/2025 11/13/2025
Login Html Skeleton 11/11/2025 11/13/2025
Student Signup html skeleton 11/11/2025 11/13/2025
tutor signup html skeleton 11/11/2025 11/13/2025
consolidated CSS file 11/11/2025 11/13/2025
Tested PM2 Redeploy 11/11/2025 11/13/2025
Email Verification html skeleton 11/11/2025 11/13/2025
Messaging html Skeleton 11/17/2025
booking html skeleton 11/17/2025 1/13/2025
clean up the repo
Beautified index.html and aboutp
Messaging backend implementat
Tutor dashboard
finish up messaging
Task Bank
William's Tracker
Jamie's Tracker
Uchenna's Tracker
Randy's Tracker
Ethan's Tracker

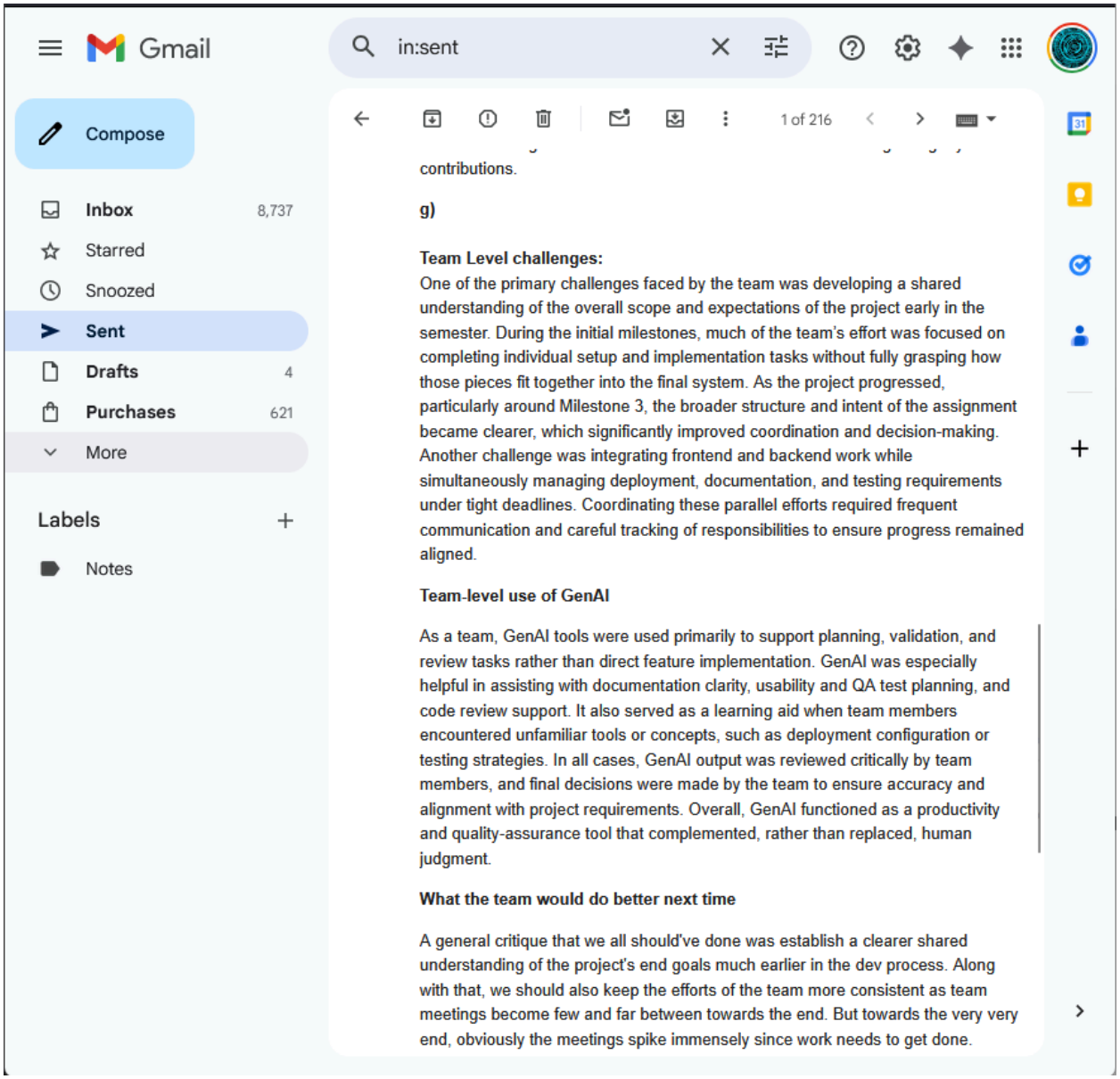
8. Team Member Self-Assessment and Contributions

William's Self assessment and contributions + team lead section g)




Gmail
in:sent
Compose
Inbox 8,737
Starred
Snoozed
Sent
Drafts 4
Purchases 621
More
Labels
Notes
1 of 216
Team Member Self-Assessment and Contributions
Summarize this email
William Wong <williamw559... 7:52 PM (3 minutes ago)
to cype1650, ethanniu2003, jameschristianbarr, Randychen168668, Uchennad04
a) Contributions to the team project and teamwork
My role on the team was Team Lead. My primary contributions focused on project coordination, deployment setup, and ensuring milestone compliance across the team. I was responsible for setting up and managing the deployment environment using AWS EC2 and PM2, verifying that the application was accessible and stable for demos and grading, and helping troubleshoot deployment-related issues. I also contributed to both frontend and backend integration tasks as needed, assisted teammates with clarifying requirements, and reviewed project documentation to ensure clarity, consistency, and alignment with course rubrics. In addition, I coordinated final Milestone 5 preparation, ensured all required sections were completed, and helped guide the team through ambiguity in milestone expectations.
b) GitHub contributions
I made 67 commits to the team GitHub repository. These commits reflect a combination of deployment configuration, integration fixes, incremental feature updates, and adjustments made to align the implementation with milestone requirements and feedback. My commit count also reflects my role as Team Lead, where responsibilities extended beyond coding to include coordination, documentation review, and overall project management.
c) Main challenges encountered
One of the main challenges I experienced during this project was learning how to use AWS, configuring the PM2 CLI for deployment, and contributing to frontend-


Gmail
in:sent
Compose
Inbox 8,737
Starred
Snoozed
Sent
Drafts 4
Purchases 621
More
Labels
Notes
1 of 216
related tasks outside my original comfort zone. While I had prior experience with web development, setting up and managing a live deployment environment introduced a significant learning curve, especially when troubleshooting issues related to EC2 configuration, process management, and environment setup. Balancing these technical challenges alongside leadership and coordination responsibilities required additional time and effort.
d) Experience with GenAI
I used ChatGPT primarily to help bridge knowledge gaps, especially when working with AWS deployment and PM2 configuration. GenAI was useful for explaining concepts, suggesting troubleshooting steps, and helping me reason through unfamiliar tooling. However, I did not rely on it exclusively; in many cases, I still needed to consult official PM2 documentation and external articles related to AWS EC2 to fully understand and resolve issues. Overall, GenAI served as a helpful guidance and learning aid, but not a replacement for hands-on experimentation and documentation review.
e) What I would do better next time
One thing I would do better next time is spending more time early in the project understanding the overall scope and big-picture goals of the assignment. During the early milestones, particularly Milestones 0 through 2, I was personally unclear on what the end goal of the project was and focused more on completing individual setup tasks without fully understanding how they fit into the larger system. It was not until working on Milestone 3 that the overall structure and intent of the assignment became clear to me. In hindsight, taking more time early on to understand the full lifecycle of the project would have helped me plan more effectively and guide the team with greater clarity.
f) Additional information for instructors
There is nothing additional I feel the instructors need to know regarding my contributions.
g)
Team Level challenges:
One of the primary challenges faced by the team was developing a shared



Randy's Self Assessment and Contribution

Uchenna’s Self assessment and contributions

Uchenna's Self Assessment



Uchi <uchennad04@gmail.com> 7:38 PM (29 minutes ago)

to williamw55955, cype1650, ethanniu2003, jameschristianbarr, Randychen168668

A.) Backend Lead

- Created the original server and credentials before they were remade later one
- Created rough data base
- Created list of functional requirements
- Created Student Notes function
- Created Tutor Availability
- Added some css styling to be reused

B.) 13 Submissions

- Would push full functions at once after testing on local server

C.)

The biggest challenges I encountered were mostly around understanding the routes and controllers in relation to our database. When implementing a function I would get most of it created in one go because the html and formatting was understandable. Any outside features, like the calendar for example, are easy to come across on the internet so they weren’t difficult. The biggest challenge came from making sure my features were actually synced up with the different columns and entities within our database so that it was fully functional.

D.)

My overall experience with GenAI was a positive one I would say. It helped a lot with getting an understanding of what I was trying to accomplish using javascript and creating things on the fly. It was also helpful for understanding errors I would have within the terminal or command prompt. It helped with some CSS features too like buttons and similar things.

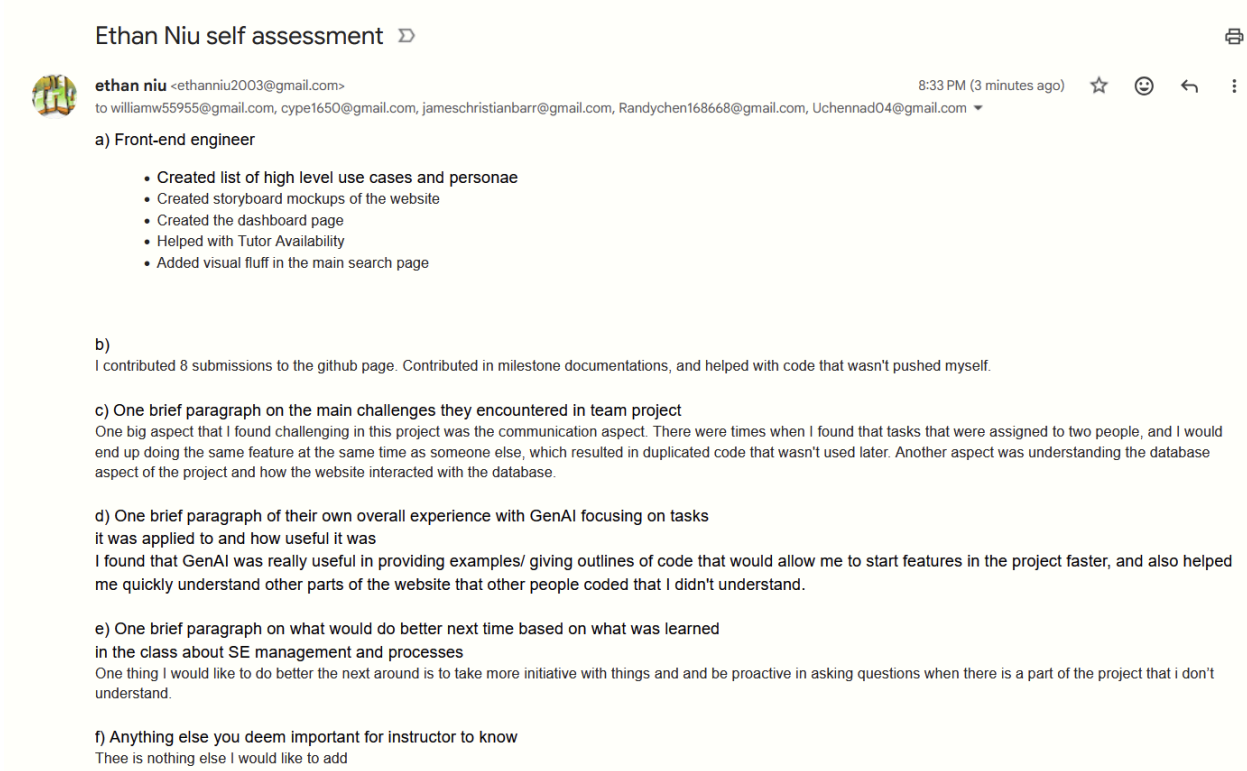
E.)

If I could do one thing better next time, I would’ve cleaned up the database myself earlier on to take some load off my teammates and just make sure I understand the routing better. It’s not that it hurt my team as much as it just wasn’t as efficient as I should have made it due to me being the backend lead.

F.)

Like I stated earlier, my github commits are pretty low count wise because I would push large chunks at once because even small issues I would test before pushing. I was doing this because I wasn’t leaving tasks uncompleted for multiple days so that my team can get their hands on the feature in an at least 90% complete state.

Ethan’s self assessment and contributions



Ethan Niu self assessment

ethan niu <ethanniu2003@gmail.com> 8:33 PM (3 minutes ago)
to williamw55955@gmail.com, cype1650@gmail.com, jameschristianbarr@gmail.com, Randychen168668@gmail.com, Uchennad04@gmail.com

a) Front-end engineer

- Created list of high level use cases and personae
- Created storyboard mockups of the website
- Created the dashboard page
- Helped with Tutor Availability
- Added visual fluff in the main search page

b)
I contributed 8 submissions to the github page. Contributed in milestone documentations, and helped with code that wasn't pushed myself.

c) One brief paragraph on the main challenges they encountered in team project
One big aspect that I found challenging in this project was the communication aspect. There were times when I found that tasks that were assigned to two people, and I would end up doing the same feature at the same time as someone else, which resulted in duplicated code that wasn't used later. Another aspect was understanding the database aspect of the project and how the website interacted with the database.

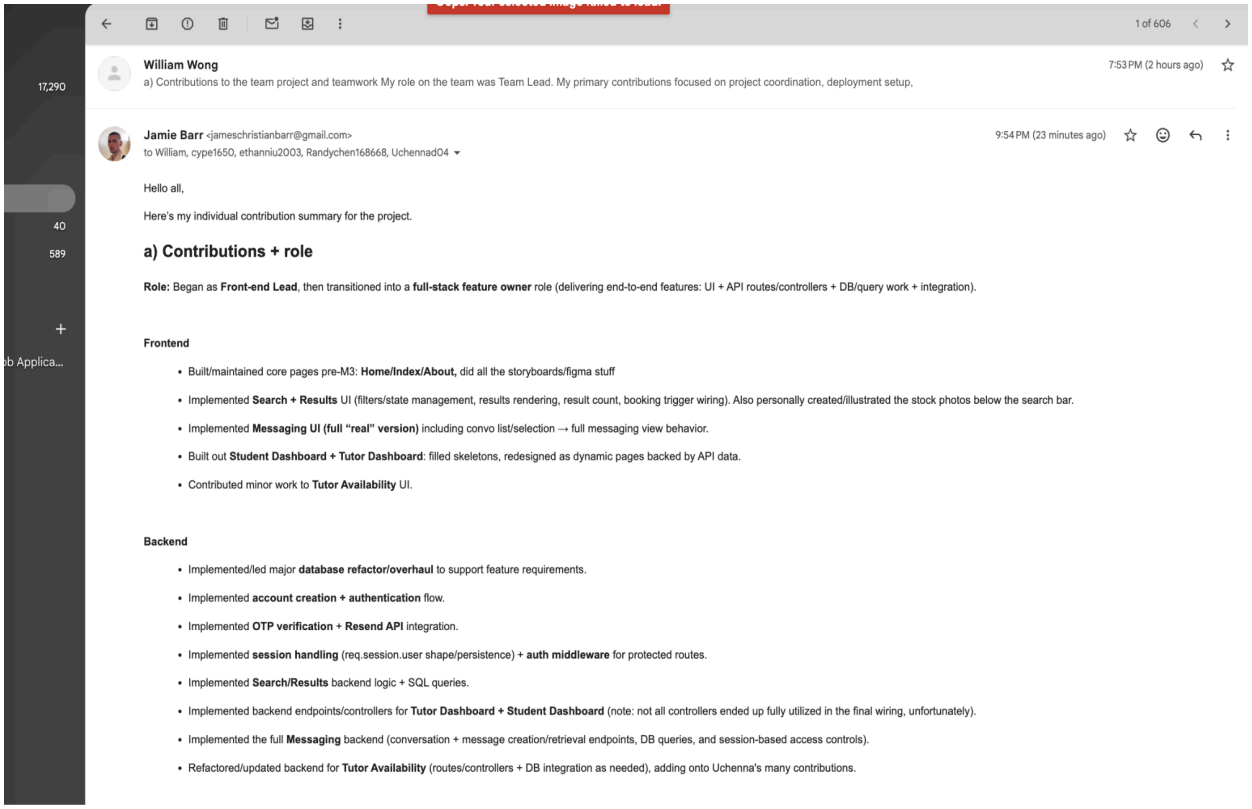
d) One brief paragraph of their own overall experience with GenAI focusing on tasks
it was applied to and how useful it was
I found that GenAI was really useful in providing examples/ giving outlines of code that would allow me to start features in the project faster, and also helped me quickly understand other parts of the website that other people coded that I didn't understand.

e) One brief paragraph on what would do better next time based on what was learned
in the class about SE management and processes
One thing I would like to do better the next around is to take more initiative with things and and be proactive in asking questions when there is a part of the project that i don't understand.

f) Anything else you deem important for instructor to know
Thee is nothing else I would like to add

Cyrus’s self assessment and contributions

Jamie Barr’s self assessment and contributions



1 of 606

William Wong 7:53 PM (2 hours ago)

a) Contributions to the team project and teamwork My role on the team was Team Lead. My primary contributions focused on project coordination, deployment setup,

Jamie Barr <jameschristianbarr@gmail.com> 9:54 PM (23 minutes ago)

to William, cype1650, ethanniu2003, Randychen168668, Uchennad04

Hello all,

Here's my individual contribution summary for the project.

a) Contributions + role

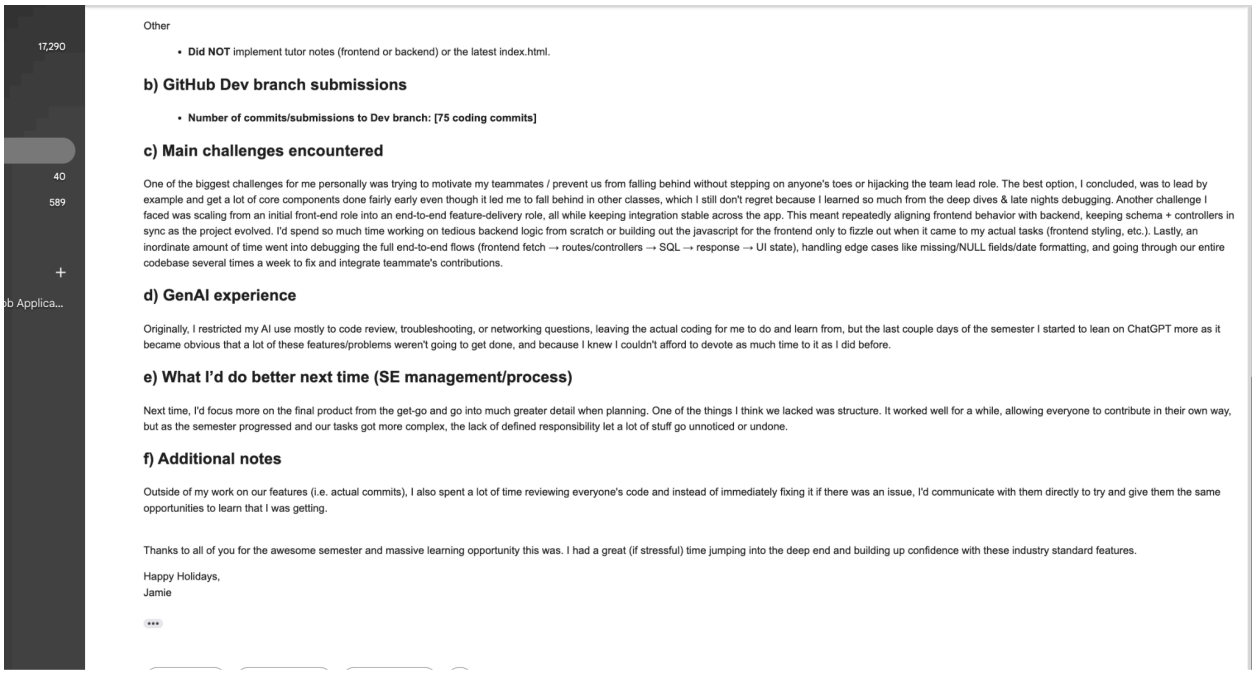
Role: Began as **Front-end Lead**, then transitioned into a **full-stack feature owner** role (delivering end-to-end features: UI + API routes/controllers + DB/query work + integration).

Frontend

- Built/maintained core pages pre-M3: **Home/Index/About,** did all the storyboards/figma stuff
- Implemented **Search + Results** UI (filters/state management, results rendering, result count, booking trigger wiring). Also personally created/illustrated the stock photos below the search bar.
- Implemented **Messaging UI (full “real” version)** including convo list/selection → full messaging view behavior.
- Built out **Student Dashboard + Tutor Dashboard**: filled skeletons, redesigned as dynamic pages backed by API data.
- Contributed minor work to **Tutor Availability** UI.

Backend

- Implemented/led major **database refactor/overhaul** to support feature requirements.
- Implemented **account creation + authentication** flow.
- Implemented **OTP verification + Resend API** integration.
- Implemented **session handling** (req.session.user shape/persistence) + **auth middleware** for protected routes.
- Implemented **Search/Results** backend logic + SQL queries.
- Implemented backend endpoints/controllers for **Tutor Dashboard + Student Dashboard** (note: not all controllers ended up fully utilized in the final wiring, unfortunately).
- Implemented the full **Messaging** backend (conversation + message creation/retrieval endpoints, DB queries, and session-based access controls).
- Refactored/updated backend for **Tutor Availability** (routes/controllers + DB integration as needed), adding onto Uchenna's many contributions.



Other

- **Did NOT** implement tutor notes (frontend or backend) or the latest index.html.

b) GitHub Dev branch submissions

- **Number of commits/submissions to Dev branch: [75 coding commits]**

c) Main challenges encountered

One of the biggest challenges for me personally was trying to motivate my teammates / prevent us from falling behind without stepping on anyone's toes or hijacking the team lead role. The best option, I concluded, was to lead by example and get a lot of core components done fairly early even though it led me to fall behind in other classes, which I still don't regret because I learned so much from the deep dives & late nights debugging. Another challenge I faced was scaling from an initial front-end role into an end-to-end feature-delivery role, all while keeping integration stable across the app. This meant repeatedly aligning frontend behavior with backend, keeping schema + controllers in sync as the project evolved. I'd spend so much time working on tedious backend logic from scratch or building out the javascript for the frontend only to fizzle out when it came to my actual tasks (frontend styling, etc.). Lastly, an inordinate amount of time went into debugging the full end-to-end flows (frontend fetch → routes/controllers → SQL → response → UI state), handling edge cases like missing/NULL fields/date formatting, and going through our entire codebase several times a week to fix and integrate teammate's contributions.

d) GenAI experience

Originally, I restricted my AI use mostly to code review, troubleshooting, or networking questions, leaving the actual coding for me to do and learn from, but the last couple days of the semester I started to lean on ChatGPT more as it became obvious that a lot of these features/problems weren't going to get done, and because I knew I couldn't afford to devote as much time to it as I did before.

e) What I’d do better next time (SE management/process)

Next time, I'd focus more on the final product from the get-go and go into much greater detail when planning. One of the things I think we lacked was structure. It worked well for a while, allowing everyone to contribute in their own way, but as the semester progressed and our tasks got more complex, the lack of defined responsibility let a lot of stuff go unnoticed or undone.

f) Additional notes

Outside of my work on our features (i.e. actual commits), I also spent a lot of time reviewing everyone's code and instead of immediately fixing it if there was an issue, I'd communicate with them directly to try and give them the same opportunities to learn that I was getting.

Thanks to all of you for the awesome semester and massive learning opportunity this was. I had a great (if stressful) time jumping into the deep end and building up confidence with these industry standard features.

Happy Holidays,
Jamie